

Tecniche e applicazioni biometriche

Fabio Scotti

fabio.scotti@unimi.it

Università degli Studi di Milano

Lezione

Deep learning per

applicazioni biometriche

Fabio Scotti
fabio.scotti@unimi.it

Università degli Studi di Milano



Outline

1. Deep learning VS classical neural networks
2. Convolutional neural networks
 1. Configurazione
 2. Allenamento
 3. Tuning
3. Autoencoders e altri modelli “deep”
4. Esempi di applicazione

Nota: in queste due lezioni si affronteranno le **basi** per capire come funzionano, come configurare ed allenare i modelli di deep learning.

La vera applicazione dei modelli sarà presentata nelle **lezioni successive** fra gli esempi applicativi e tecniche di risoluzione.

Riassumendo

3 concetti fondamentali del ML per la biometria

- **E_train → 0!**

I modelli di ML, e specialmente le reti neurali di tipo Deep, possono apprendere in modo esatto i dati di allenamento (Errore = 0 sul train set)

- **CV**

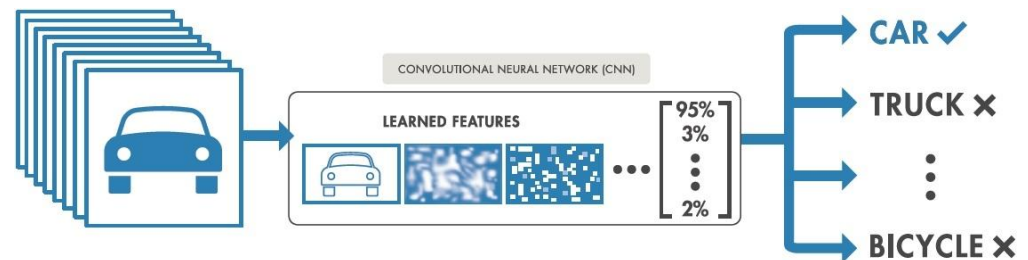
Per questo motivo usiamo la procedura di **cross-validation**, ovvero creiamo SEMPRE un insieme di dati di TRAINING (train test) e uno di VALIDAZIONE (spesso chiamato test set).

- Nessun dato in validazione deve essere duplicato in training

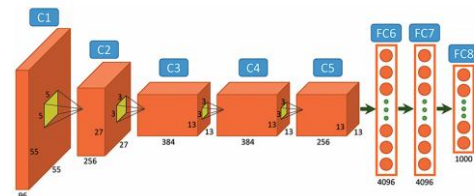
- **LNPO**

Nel caso dei sistemi biometrici, la semplice cross-validazione non basta e invece **usiamo il sistema Leave N People Out (LNPO)**

Convolutional Neural Networks (CNNs)



Come useremo le CNN nel resto del corso?

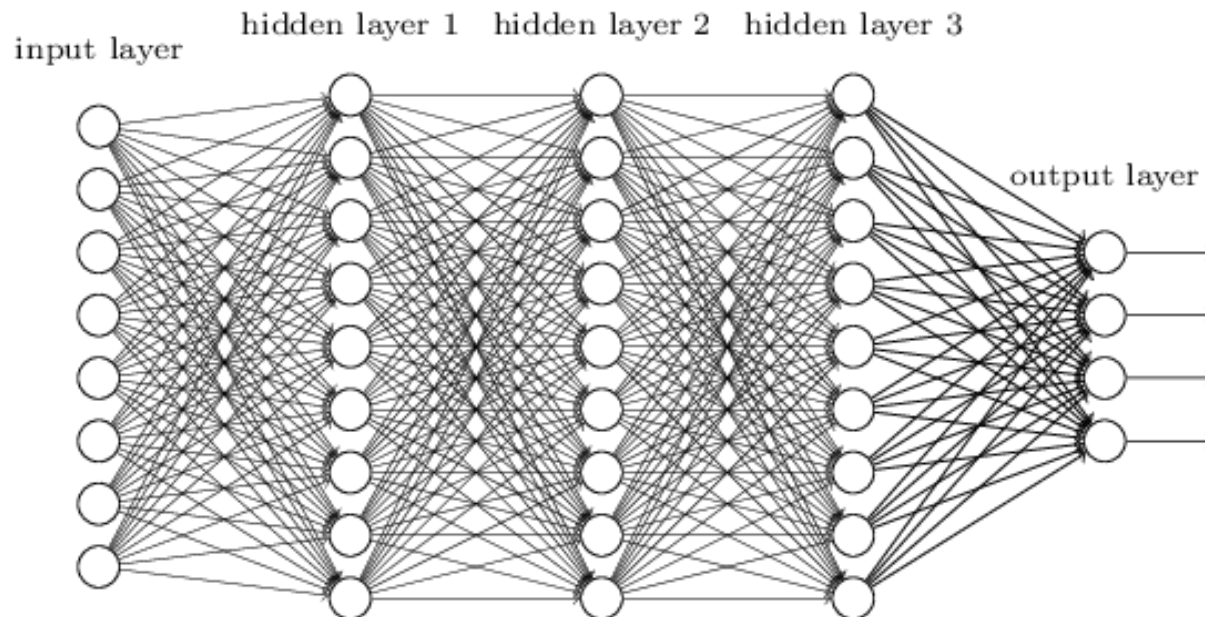


- Prima capiamo
 - **come sono fatte** le CNN
 - come si **allenano** per intero o in alcune parti
- Poi vediamo **come adattare** a nuovi problemi singoli (es. la *face detection*)
- Useremo
 - modelli già addestrati scaricabili
 - Modelli specifici da potenti **librerie pubbliche**
- Vedremo le CNN come **estrattore di caratteristiche** per adattare come strumenti alle nostre prossime diverse applicazioni!



Partiamo da una piccola rete totalmente connessa

- Sappiamo che è utile partire da un modello piccolo.
 - Le reti neurali di piccole dimensioni (con pochi neuroni e strati) possono già dimostrare la capacità di apprendere caratteristiche rilevanti dai dati.
- Da questo modello completamente connesso, abbiamo davvero bisogno di tutti gli archi (collegamenti)?
- Alcuni di questi possono essere condivisi?



Caso applicativo: classificare una immagine

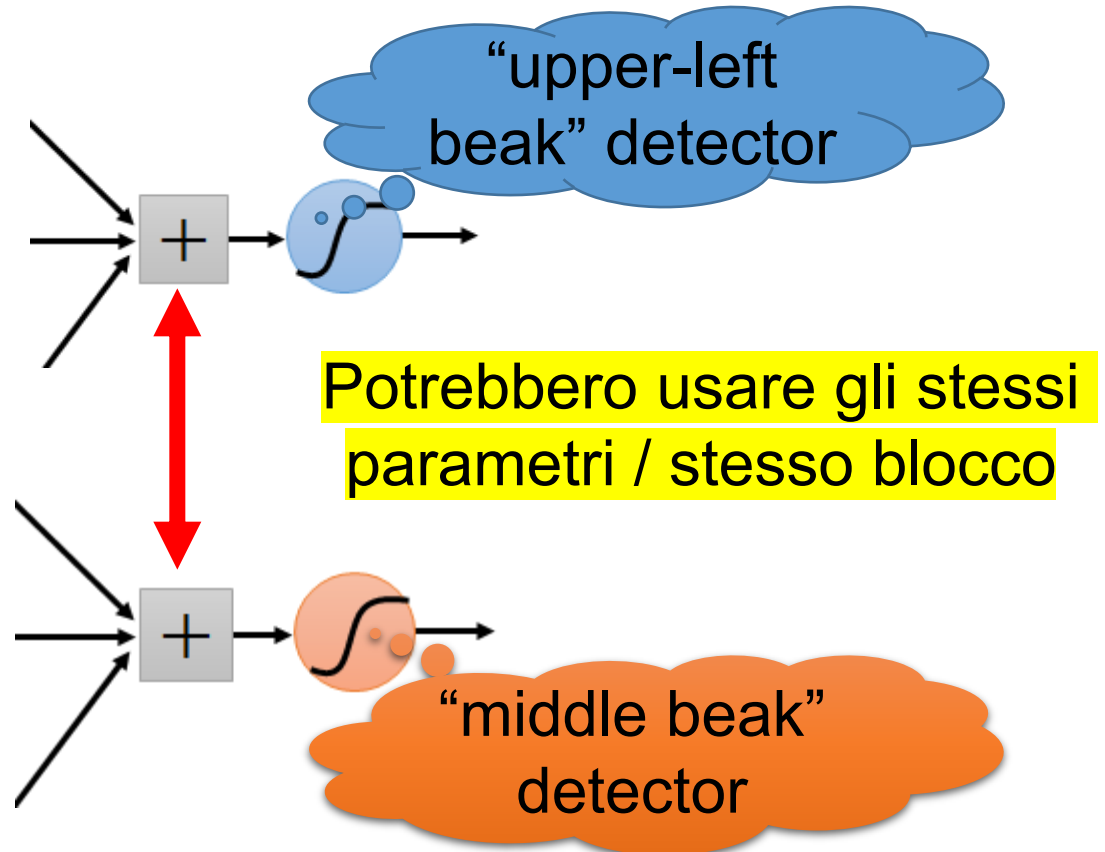
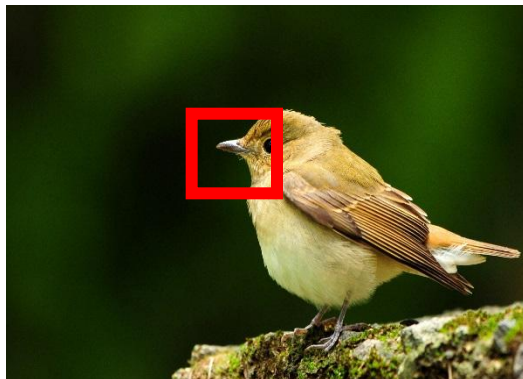
- Processare una immagine con una rete neurale significa imparare tutte le possibilità dei pixel in ingresso
→ e sono un numero astronomico!
- Alcuni pattern però sono più piccoli della intera immagine

Si può rappresentare una piccolo regione della immagine con meno parametri → capire che nel blocco abbiamo un becco



Lo stesso modello appare in diversi luoghi: può essere compresso creare una rete che lo trova ovunque in un colpo solo!

E se addestrassimo molti di questi "piccoli" rilevatori per oggetti diversi, e ogni rilevatore dovesse «scorrere» nella immagine?



Servirebbe una enorme rete piena di rilevatori in tutte le possibili posizioni!

IS4

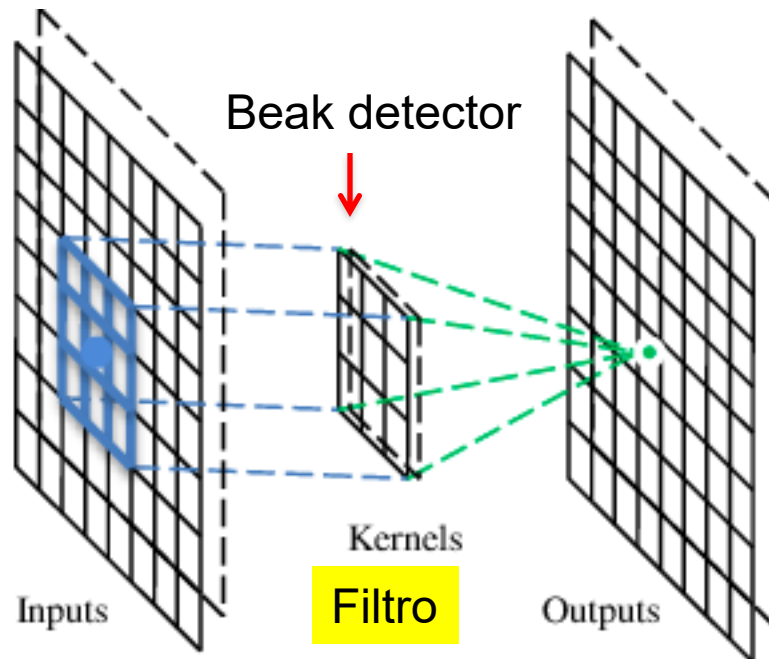
Intelligent systems for industry, supply chain and environment

- Per chi è interessato a questi argomenti tengo questo insegnamento:
[Intelligent systems for industry, supply chain and environment](#)
- Con una approccio simile all'insegnamento di «Sistemi biometrici» e «Tecniche e applicazioni biometriche», si presentano metodi, modelli e applicazioni dell'intelligenza artificiale in diversi settori e applicazioni.

Strati convoluzionali

Una CNN è una rete neurale con alcuni livelli convoluzionali (e alcuni altri livelli).

Un livello convoluzionale ha un numero di filtri che eseguono operazioni di convoluzione.



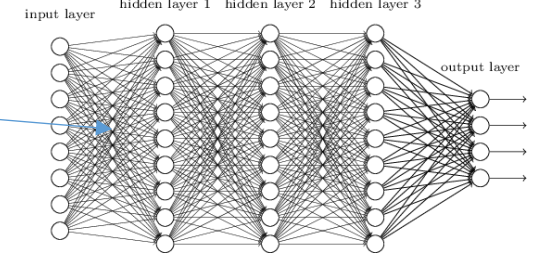
L'uscita della
convoluzione
risponde
dove
l'immagine in
ingresso è
simile al
kernel usato

Convolution kernels da imparare

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 input
image

Nelle NN trad.
erano solo i pesi



**Questi sono i parametri
che devono essere poi
fissati**

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a
small pattern (3 x 3).

Ripasso della convoluzione

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 input
image

Dot
product



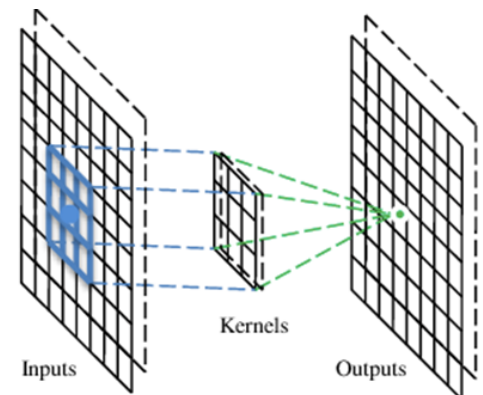
3

-1

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Vediamo in dettaglio
questa operazione



Ripasso della convoluzione

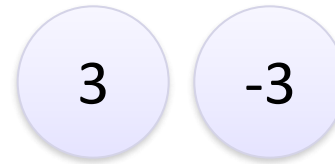
If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

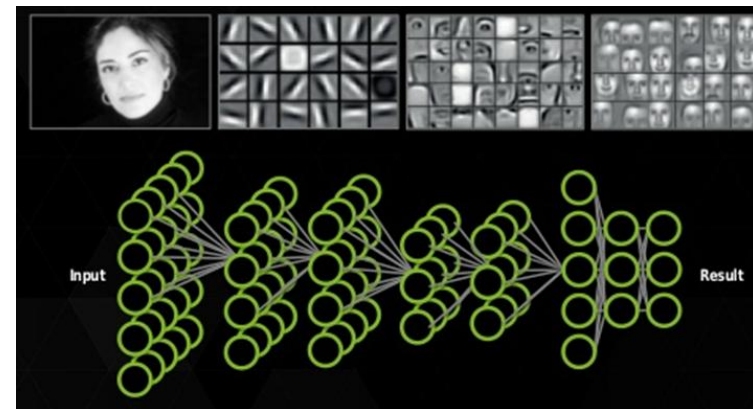
6 x 6 input
image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

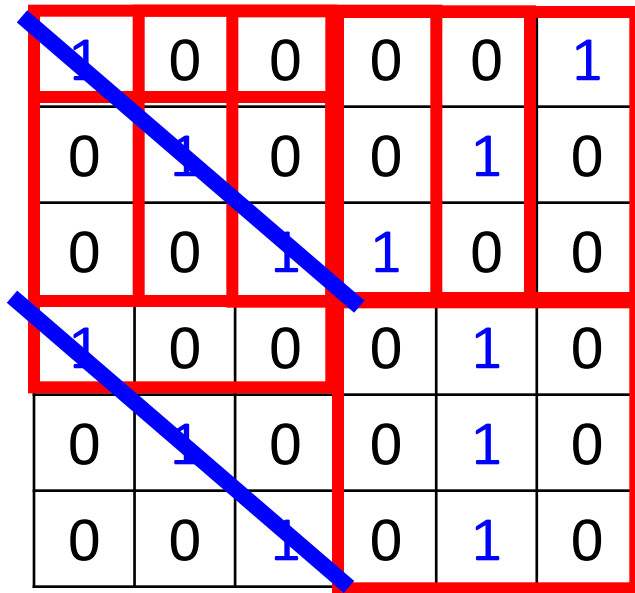


Questo potrebbe essere
uno dei tanti strati di
convoluzione della CNN



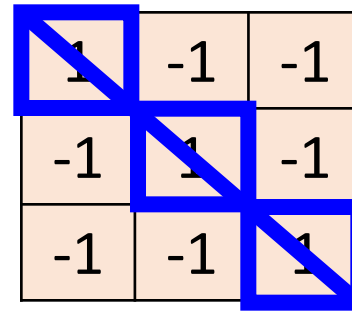
Ripasso della convoluzione

stride=1



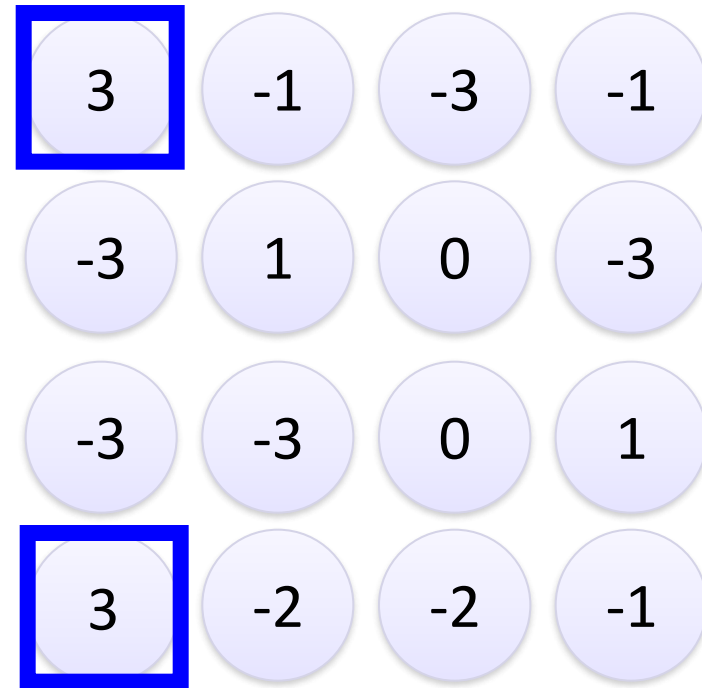
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Strato della CNN

-1	1	-1
-1	1	-1
-1	1	-1

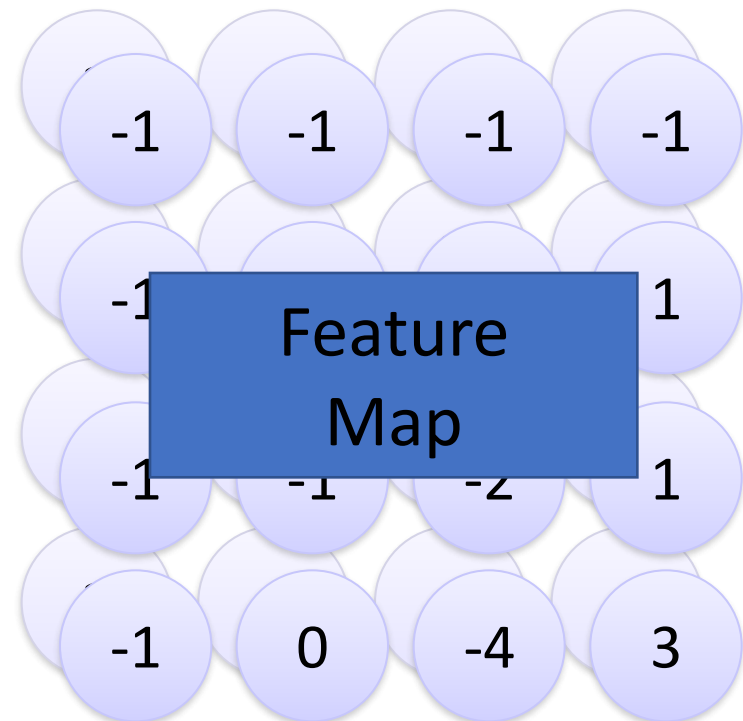
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

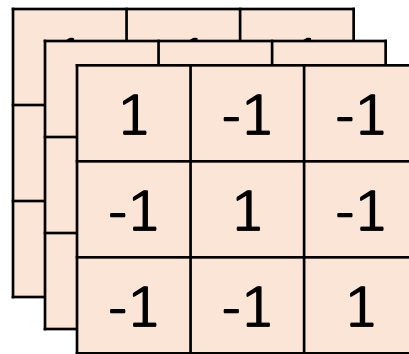
6 x 6 image

Repeat this for each filter



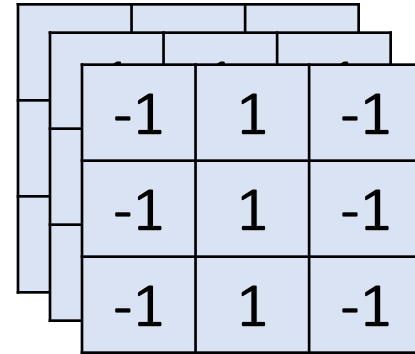
Two 4 x 4 images
Forming 2 x 4 x 4 matrix

Imagine a colori RGB 3 channels



1	-1	-1
-1	1	-1
-1	-1	1

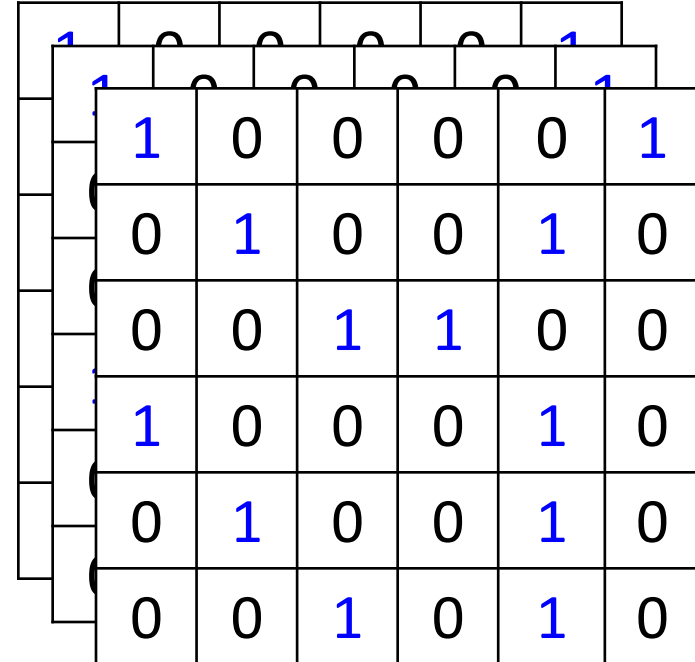
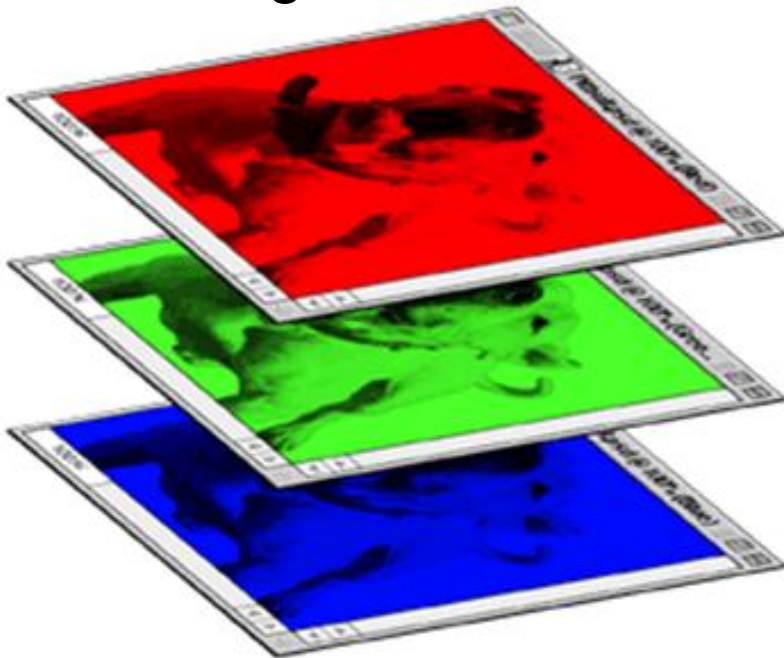
Filter 1



-1	1	-1
-1	1	-1
-1	1	-1

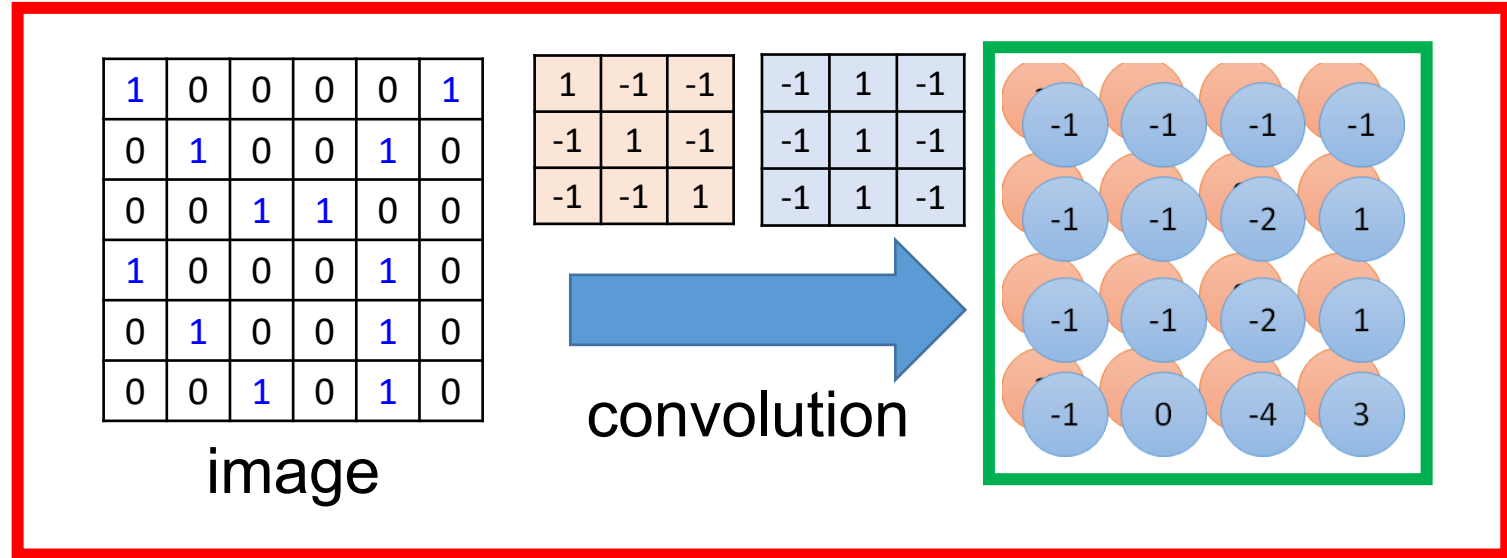
Filter 2

Color image



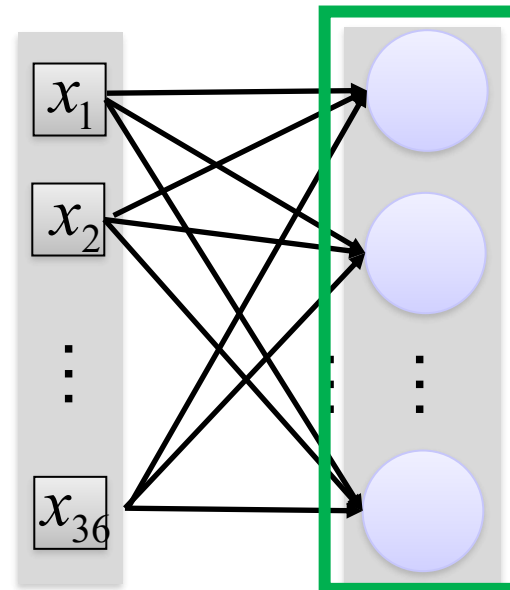
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

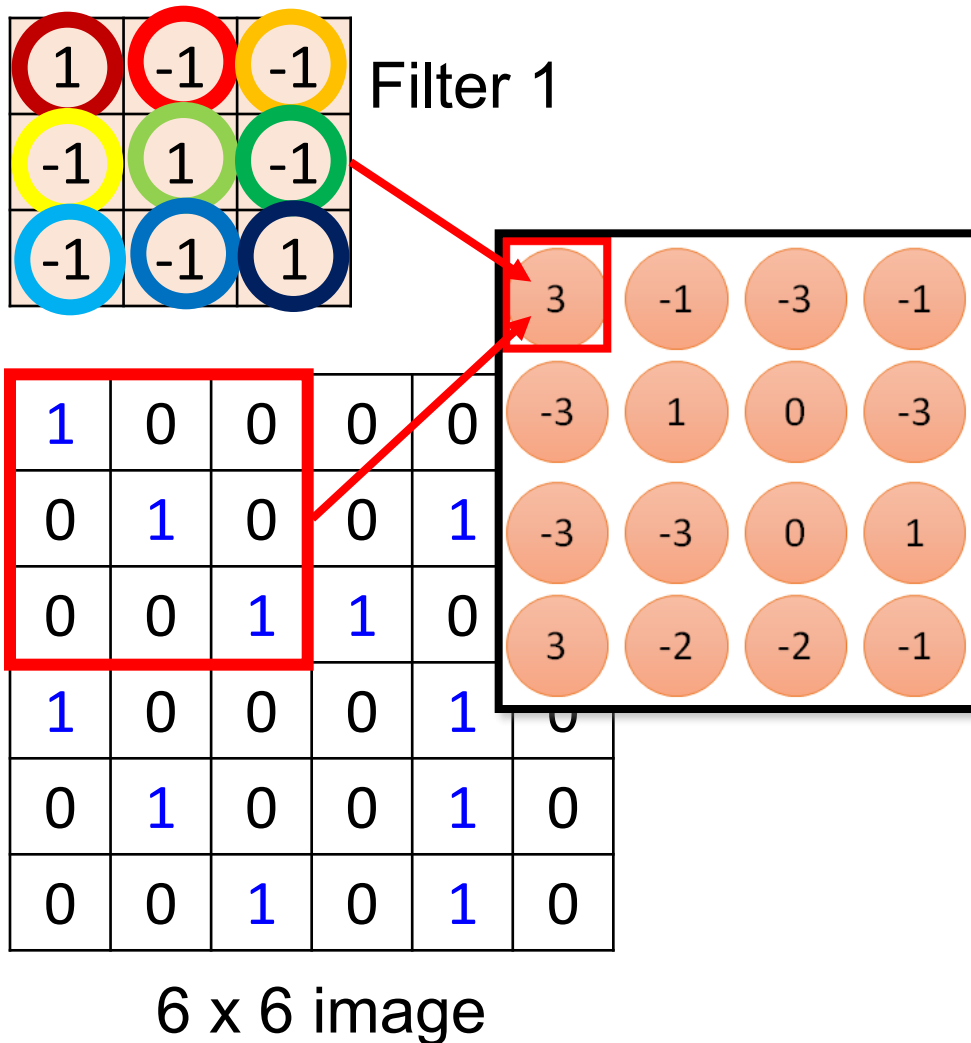
Convolution v.s. Fully Connected (guardiamo 1 strato)



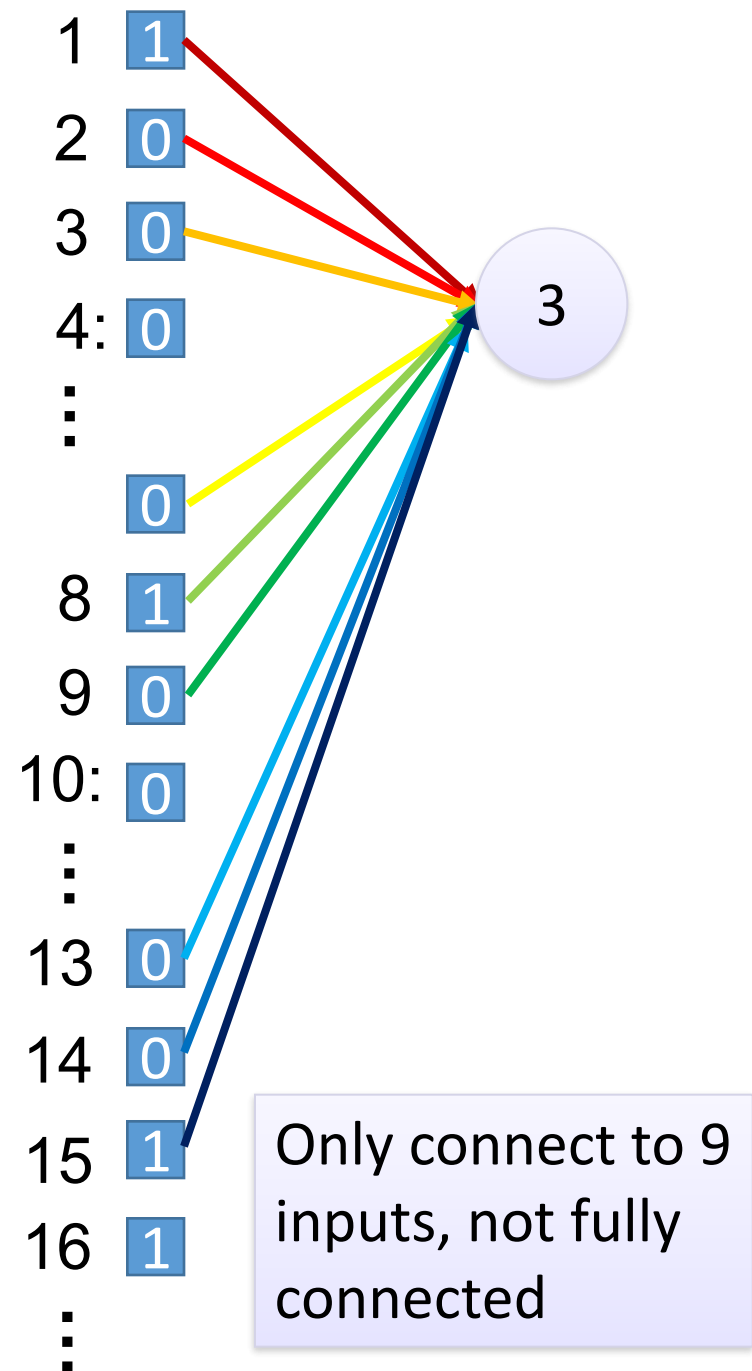
Fully-
connected

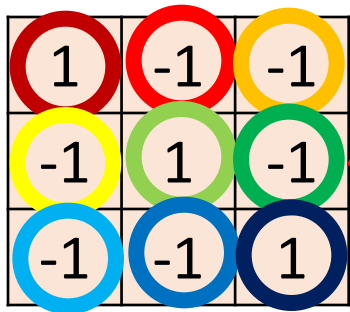
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



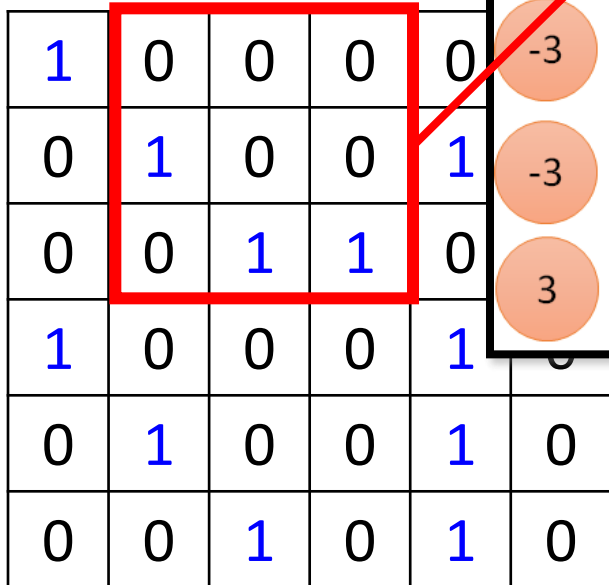


Usiamo molti
meno parametri !!

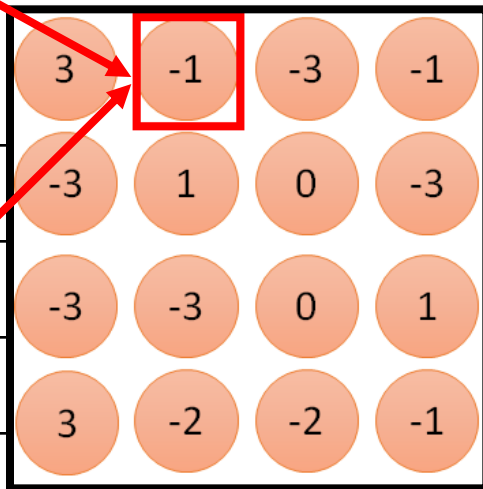




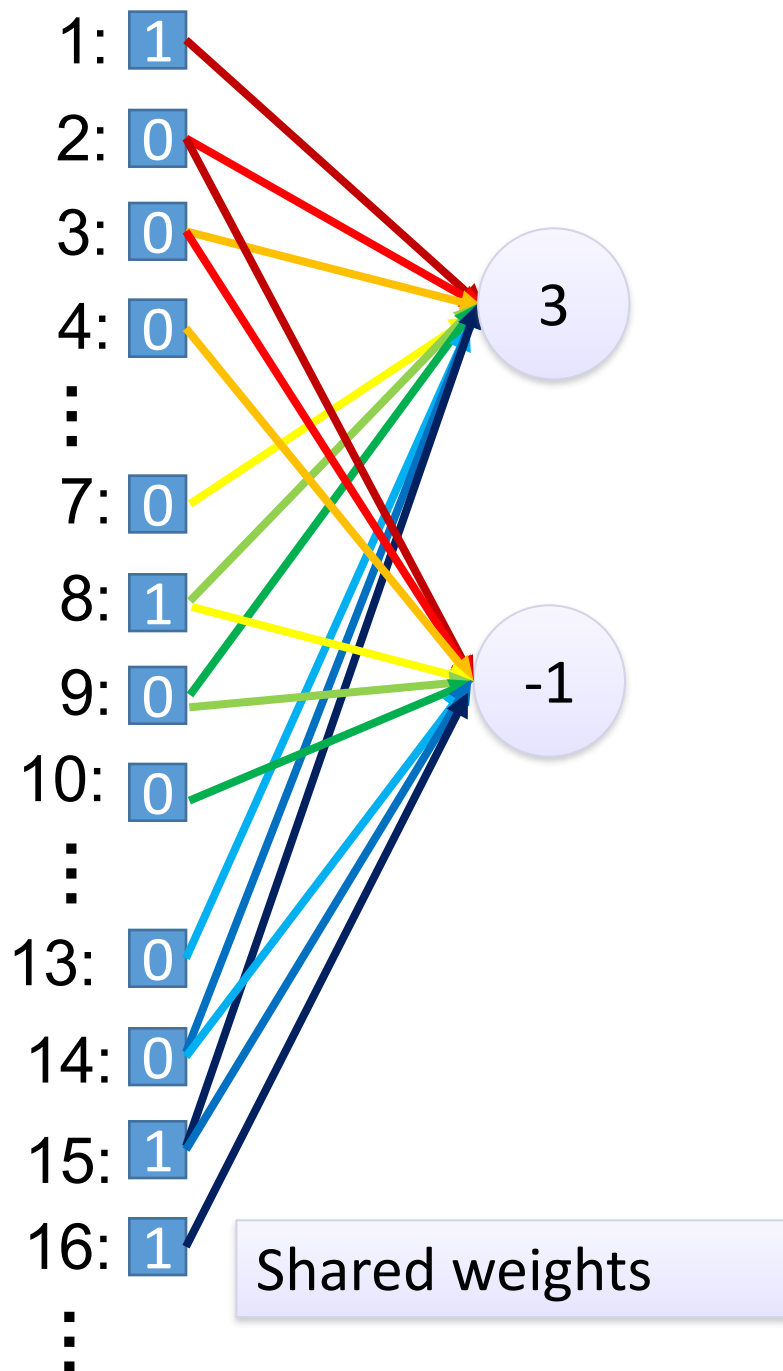
Filter 1



6 x 6 image

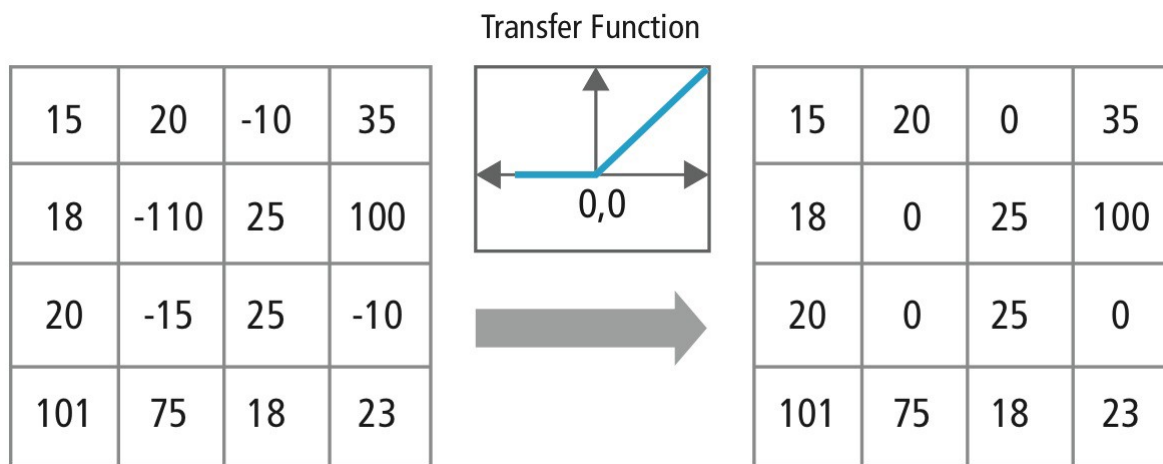
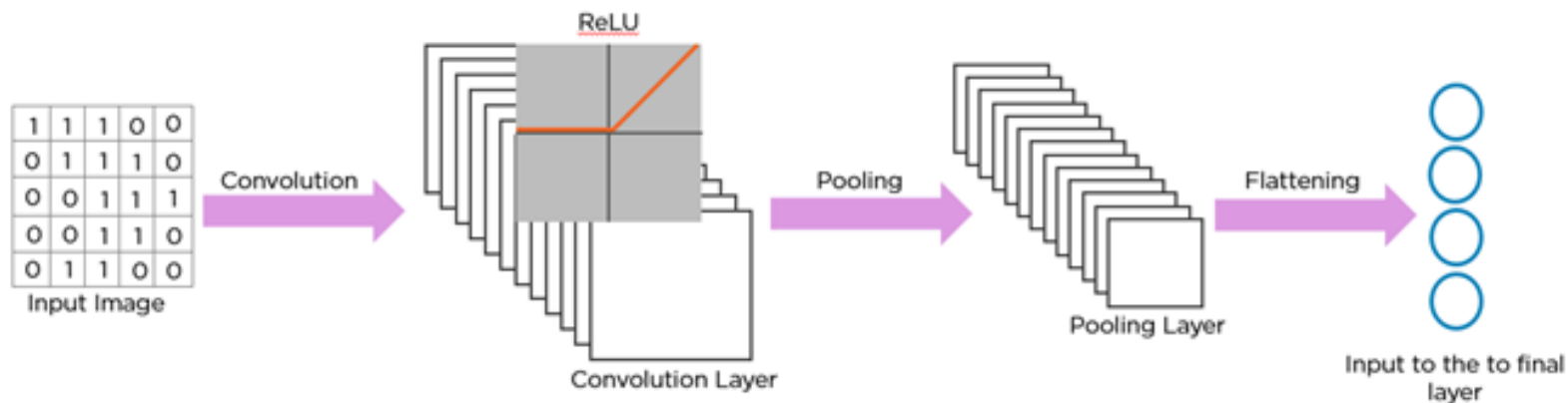


Usiamo ANCORA MENO
meno parametri !!
(riusiamo alcuni precalcoli)



ReLu

Utile per creare
gradiente nullo nella funzione
di ottimizzazione dei pesi
→ semplificazione apprendimento



Activation Functions

Sigmoid
 $\sigma(x) = \frac{1}{1+e^{-x}}$



tanh
 $\tanh(x)$



ReLU
 $\max(0, x)$



Leaky ReLU
 $\max(0.1x, x)$

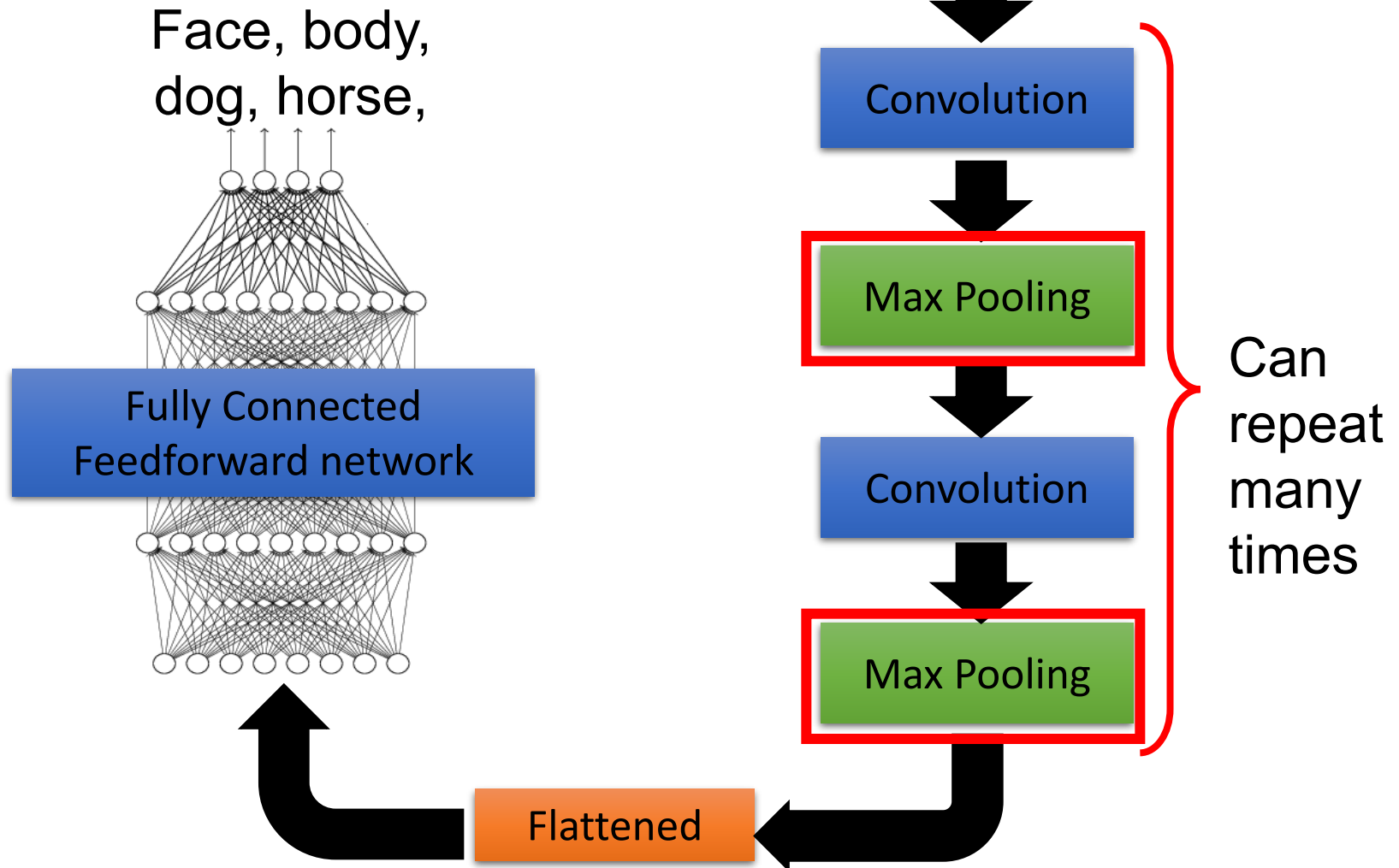


Maxout
 $\max(w_1^T x + b_1, w_2^T x + b_2)$

ELU
 $\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$

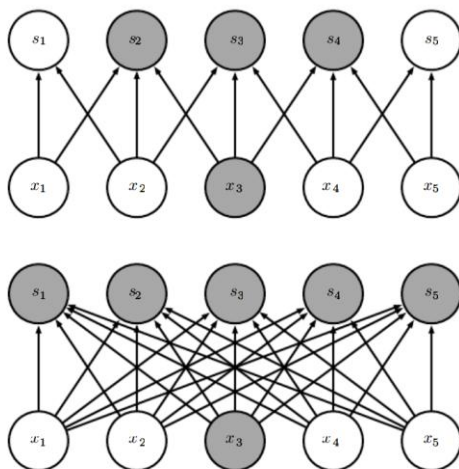


Ecco la CNN!



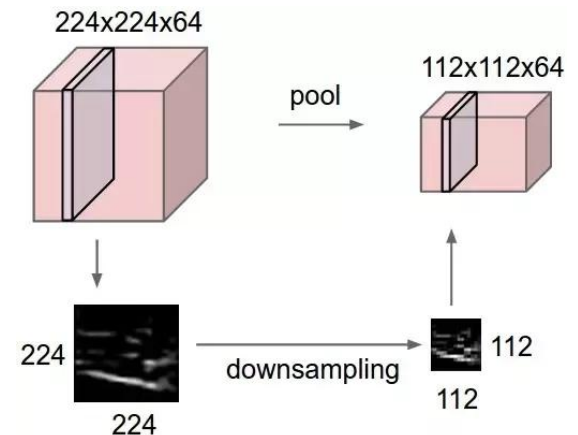
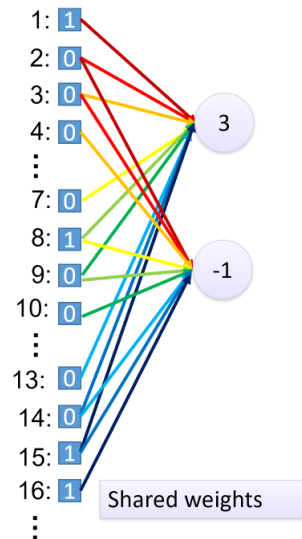
CNN $\leftrightarrow$ Fully connected network

- La CNN fa quello che fa la Fully connected network ma riesce a
 - Ridurre il numero di connessioni
 - Condividere i pre calcoli con altri neuroni
 - Attraverso il pooling riduce la complessità del problema



CNN

Fully



Perchè si adotta il “Pooling”

- Il sottocampionamento dei pixel non cambierà l'oggetto.

bird



Subsampling

bird

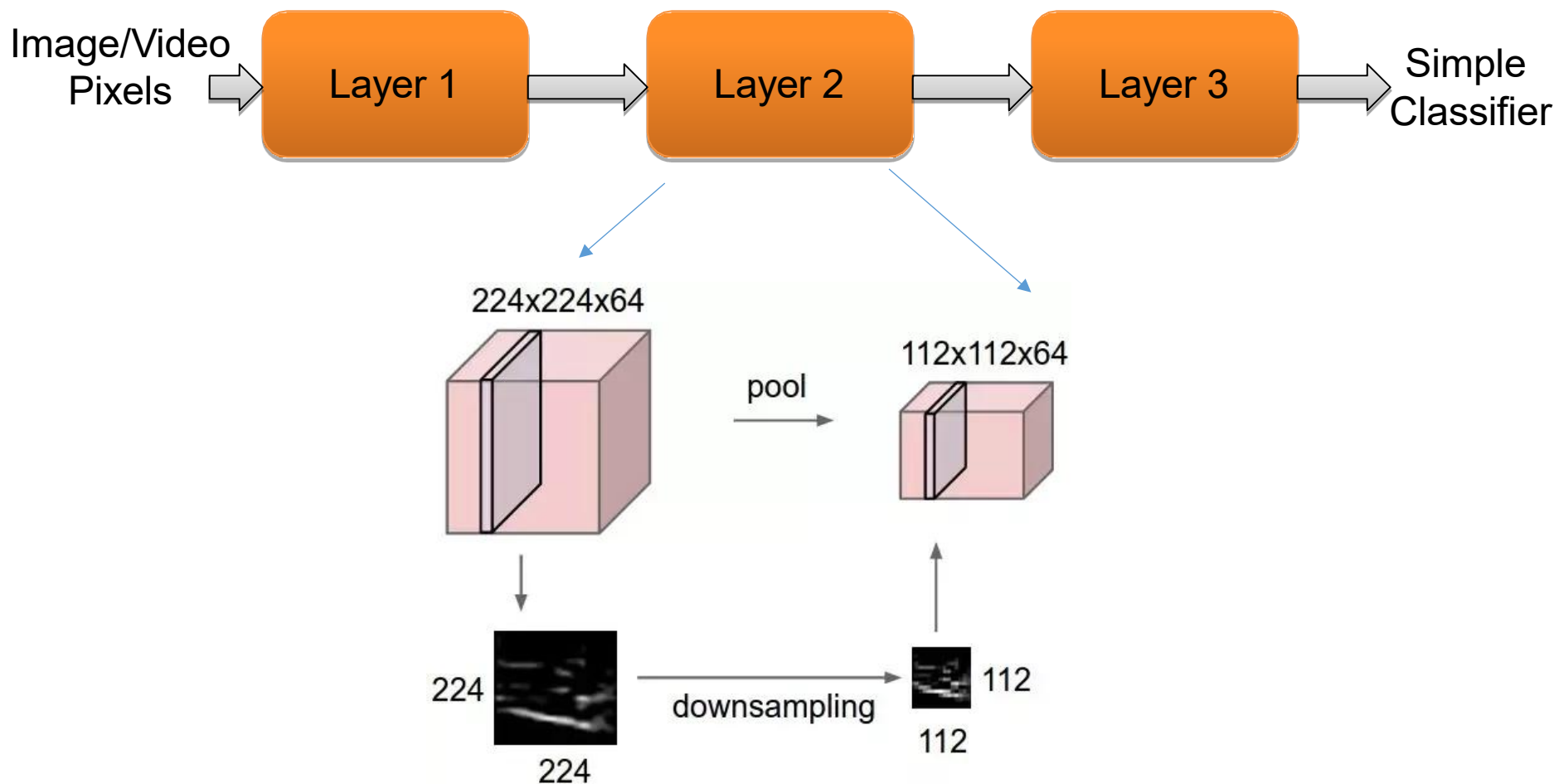


Possiamo sottocampionare i pixel per rendere l'immagine più piccola.



Serviranno meno parametri per processare l'immagine

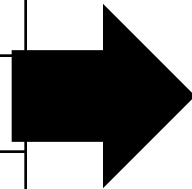
Come si realizza il pooling



Max Pooling

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

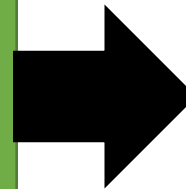
6 x 6 image



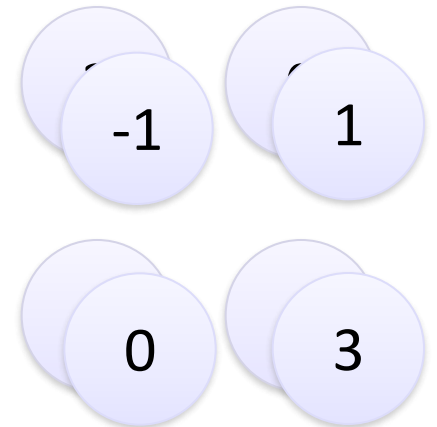
Conv



Max
Pooling



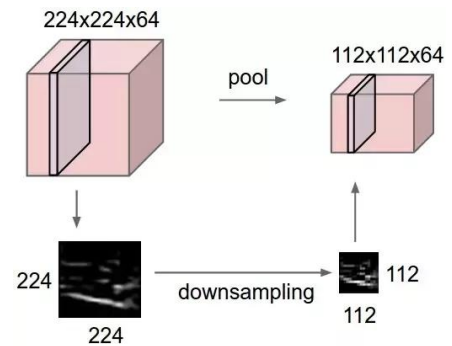
New image
but smaller



2 x 2 image

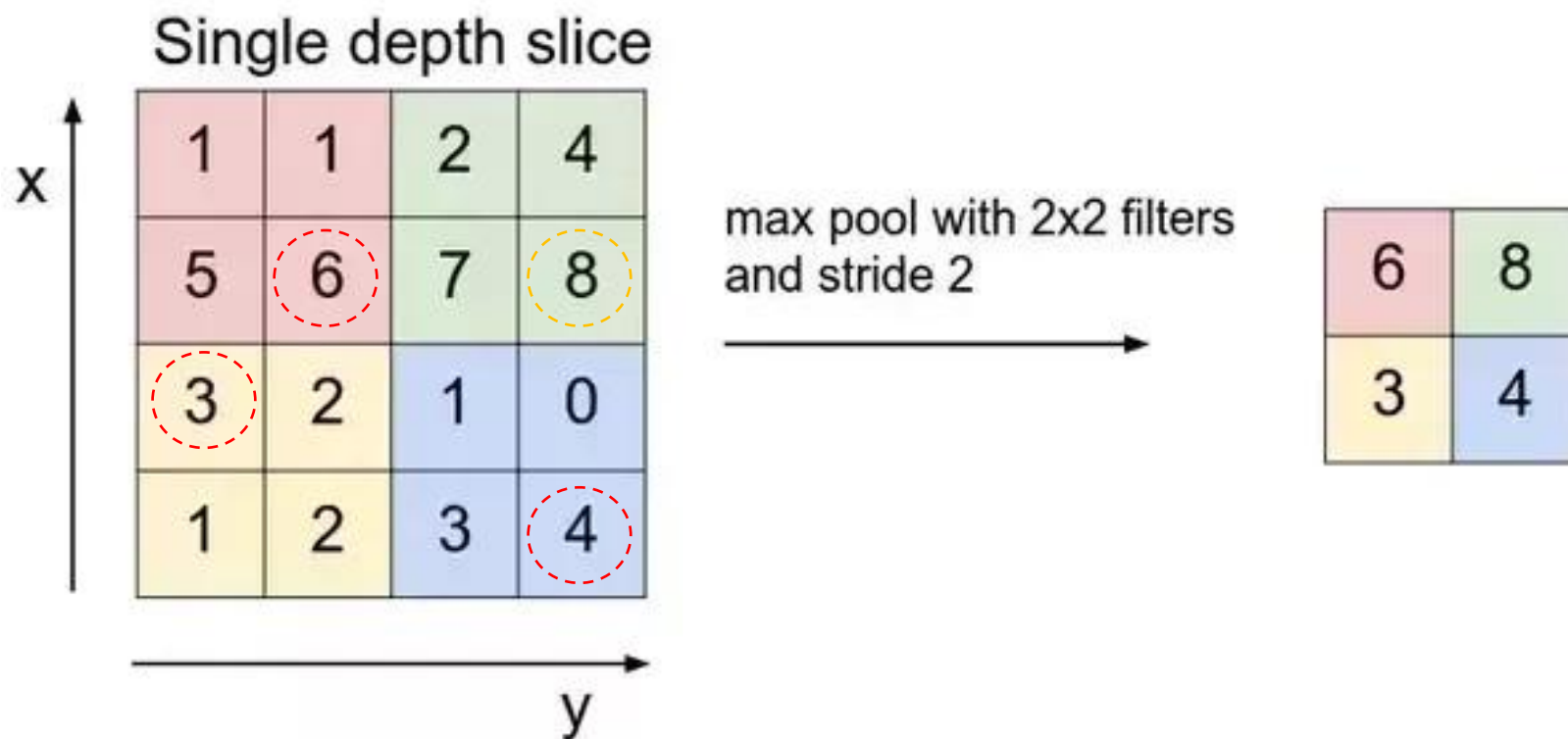
Each filter
is a channel

Sub-Sampling (Pooling)

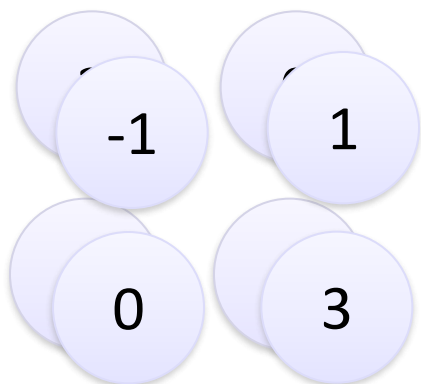


- Il sottocampionamento (Pooling) permette di ridurre il numero di caratteristiche, senza sovrapposizioni.
- Riduce la risoluzione spaziale e quindi diminuisce naturalmente l'importanza di dove esattamente è stata trovata una caratteristica, mantenendo solo la posizione approssimativa.
 - Media o Max-Pooling sono comuni tecniche utilizzate. Un pooling 2x2 comporterebbe una compressione 4:1, un 3x3 una 9:1, ecc.
- Il pooling uniforma i dati e rende i dati invarianti a piccoli cambiamenti traslazionali.
- Dopo il primo strato, ci sono sempre più mappe di caratteristiche da collegare allo strato successivo; è una decisione umana predefinita da quale delle mappe precedenti la mappa corrente riceve gli input.

Esempio di max pooling



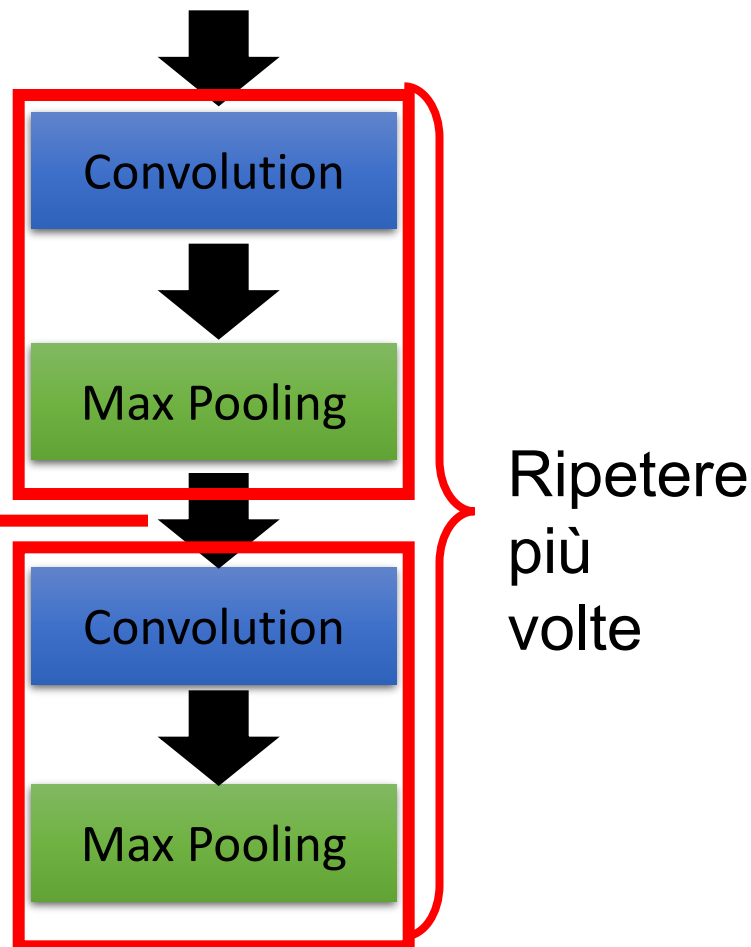
La CNN completa



Nuova immagine

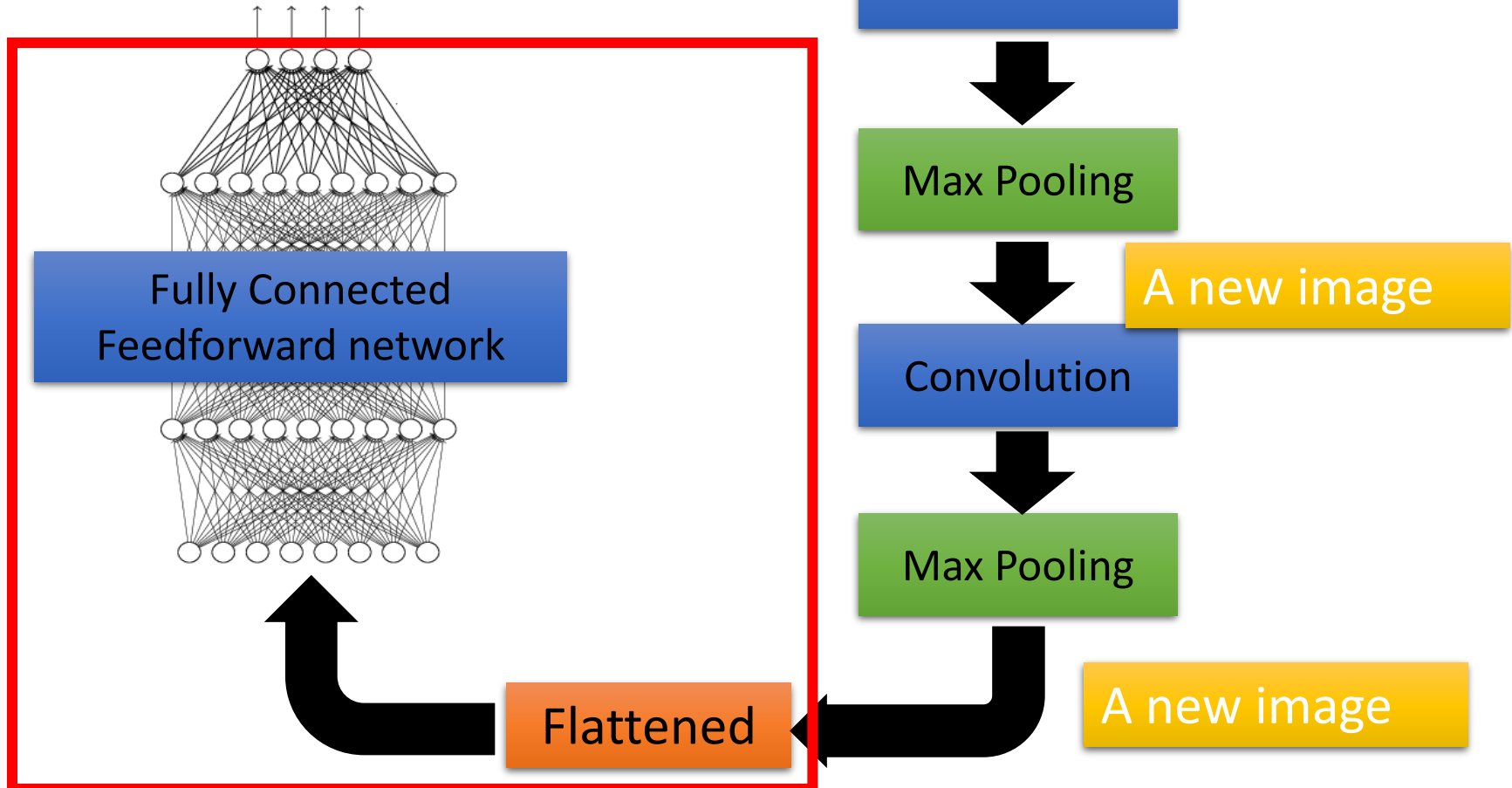
Più piccolo della originale

Il numero di canali in uscita
è uguale al Numero di filtri dello strato
* Numero di canali in ingresso

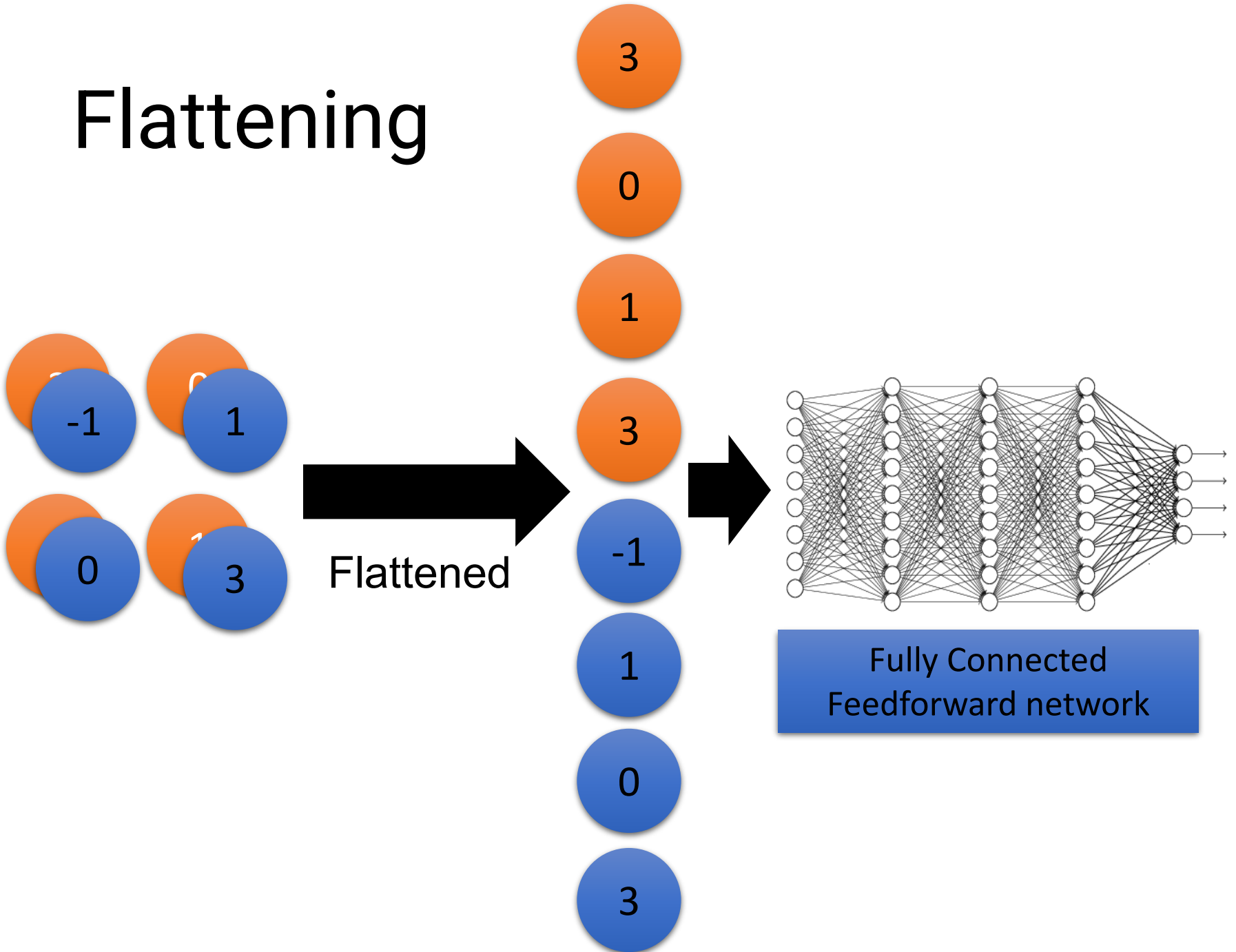


La CNN completa

cat dog



Flattening



CNN in Keras

Only modified the *network structure* and *input format* (vector -> 3-D tensor)

```
model2.add( Convolution2D( 25, 3, 3,  
                           input_shape=(28, 28, 1)) )
```

1	-1	-1
-1	1	-1
-1	-1	-1

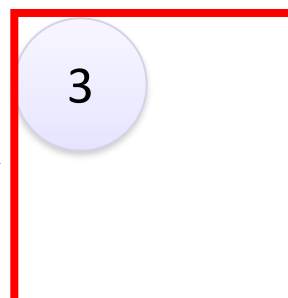
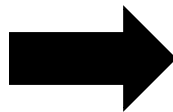
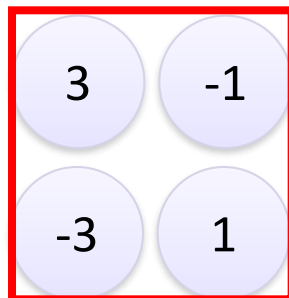
There are
25 3x3
filters.

Input_shape = (28 , 28 , 1)

28 x 28 pixels

1: black/white, 3: RGB

```
model2.add(MaxPooling2D( (2, 2) ))
```



input

Convolution

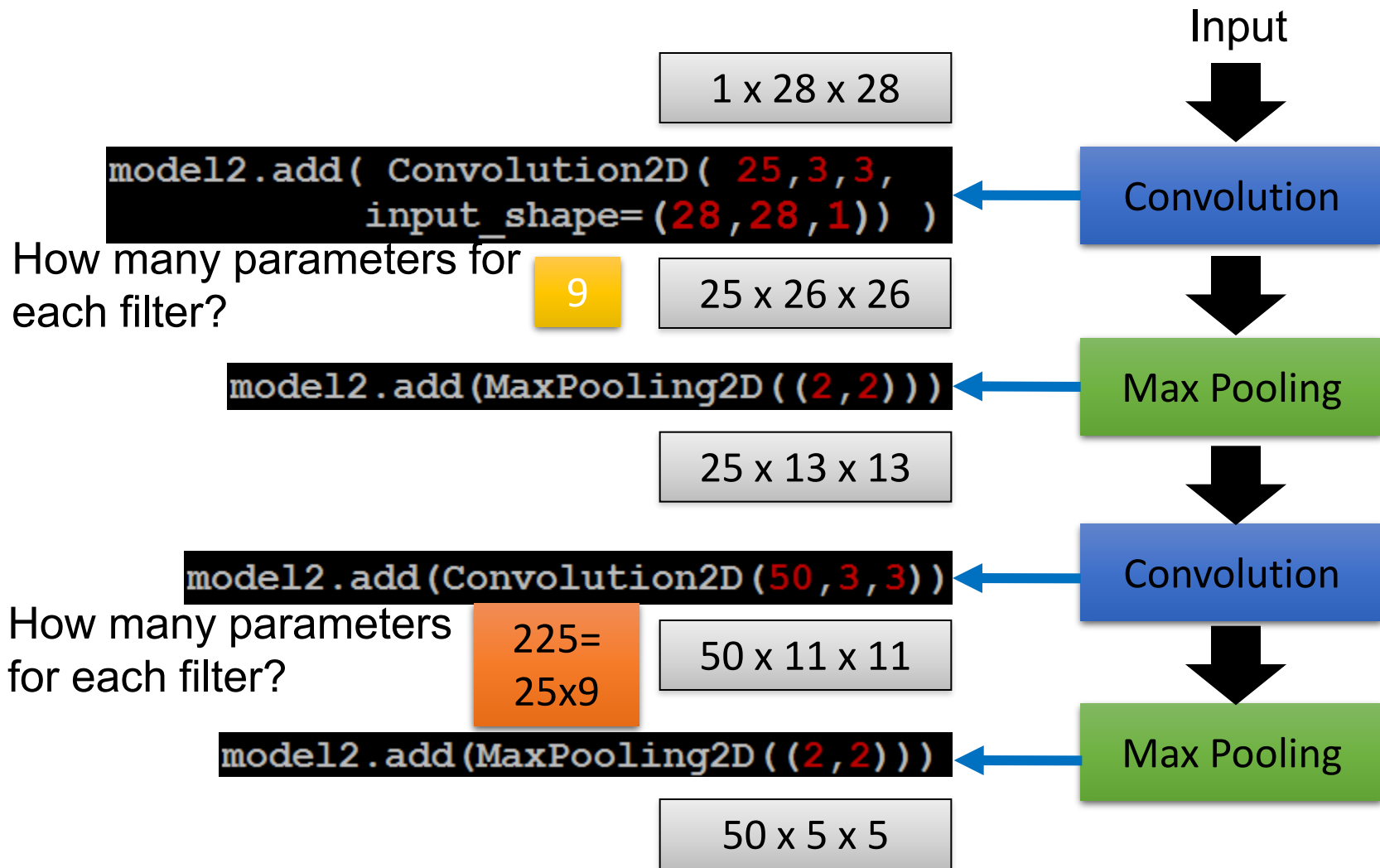
Max Pooling

Convolution

Max Pooling

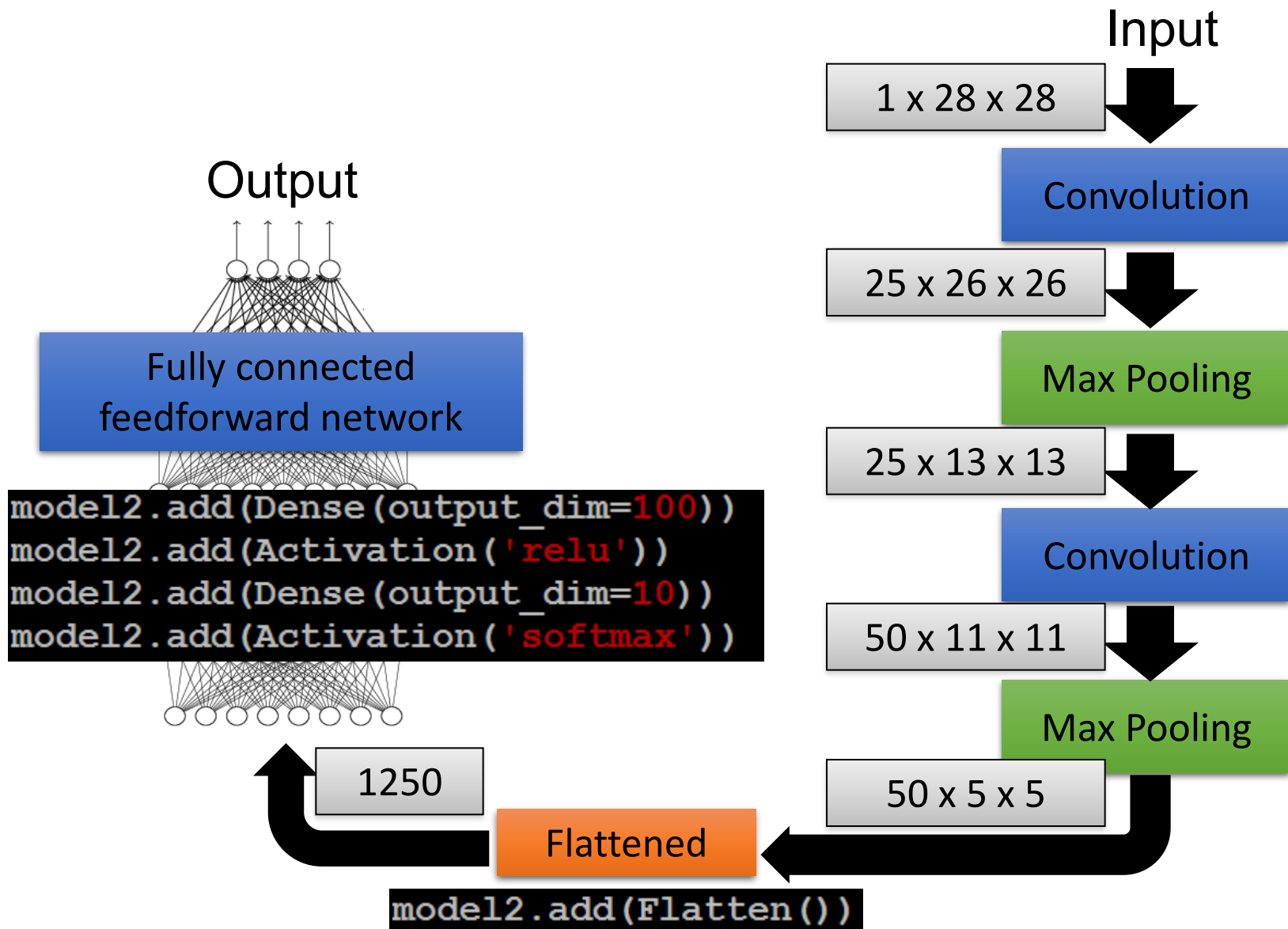
CNN in Keras

Only modified the *network structure* and *input format* (vector -> 3-D array)



CNN in Keras

Only modified the *network structure* and *input format* (vector -> 3-D array)



AlphaGo

Avere «solo» 19x19 ingressi non significa che si stia affrontando un problema semplice...



19 x 19 matrix

Black: 1

white: -1

none: 0



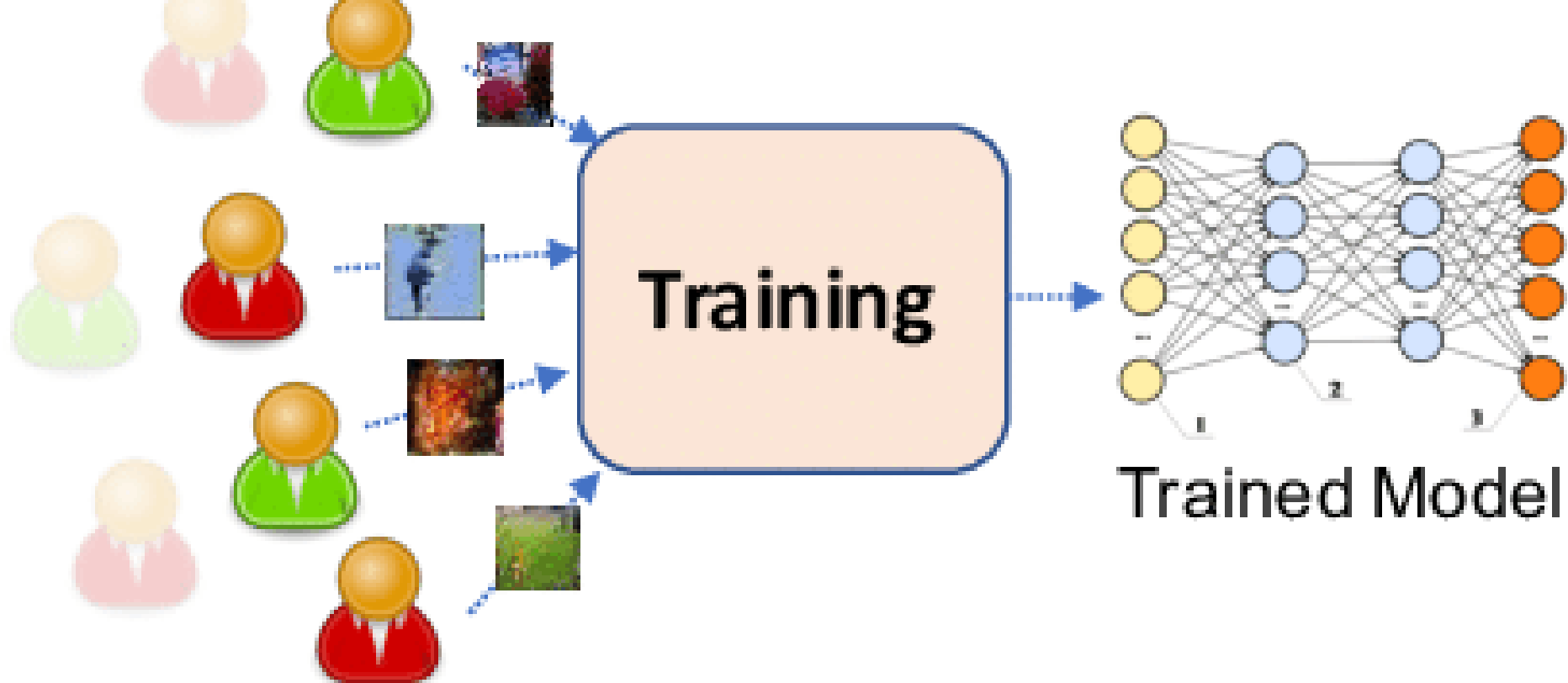
Neural
Network



Next move
(19 x 19
positions)

Fully-connected feedforward network
can be used

But CNN performs much better



Allenare reti Deep

Con metodi supervisionati e non supervisionati

Design e allenamento delle CNN

- La maggior parte dei pesi sono addestrati con la Retropropagazione (Backpropagation)
- La struttura della CNN è attualmente solitamente costruita a mano con tentativi ed errori:
 - Numero totale di strati,
 - Numero di kernel,
 - Dimensione dei kernel,
 - Dimensione dei campi di sottocampionamento (pooling),
 - A quali campi dello strato precedente connettersi.

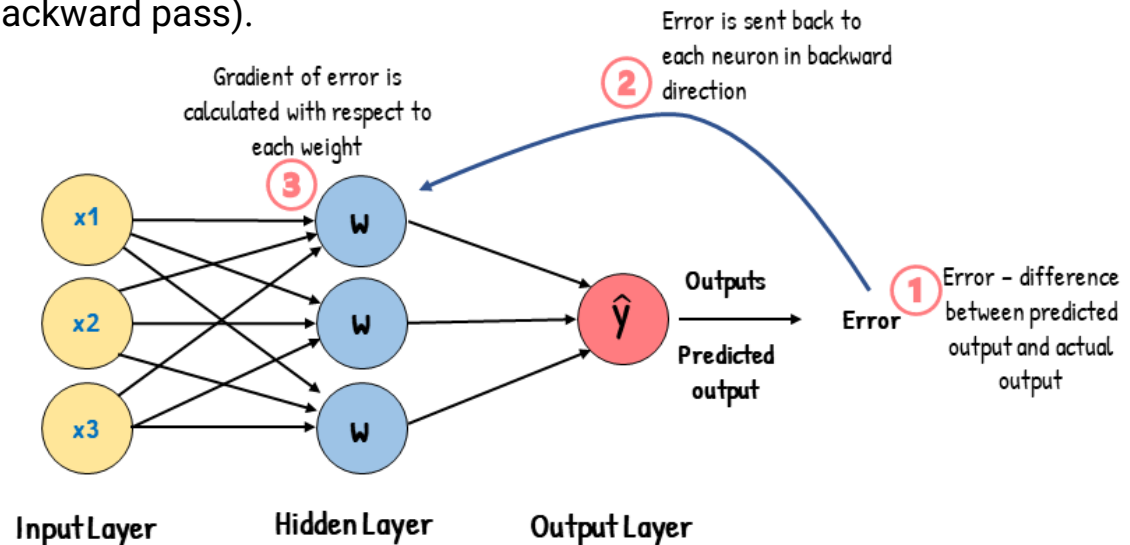
Tipicamente

- si riduce la dimensione delle mappe di caratteristiche
- si aumenta il numero di mappe di caratteristiche per gli strati successivi.



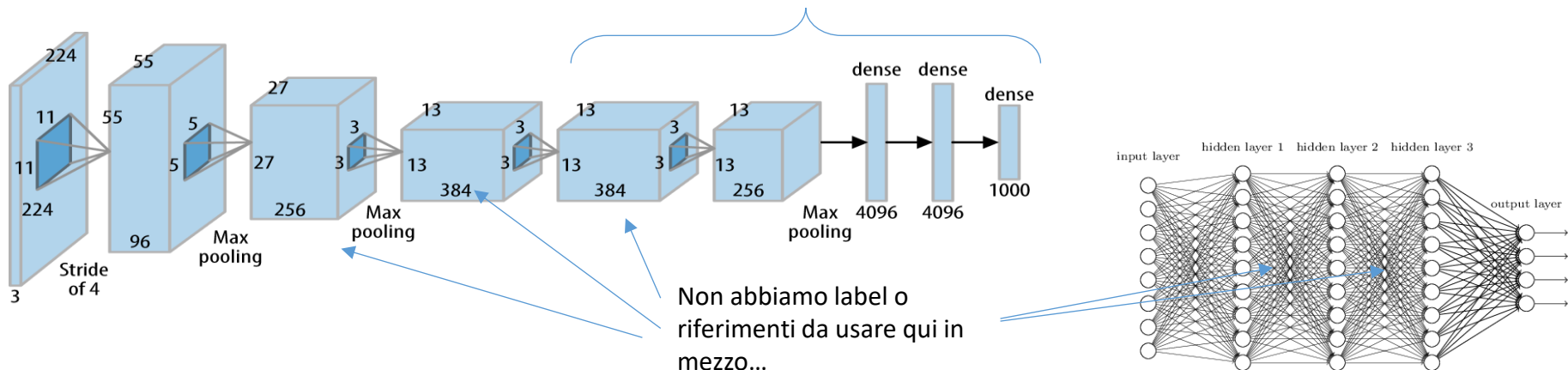
Backpropagation

- La **backpropagation** (abbreviazione di "backward propagation of errors", retropropagazione degli errori) è un algoritmo fondamentale utilizzato nell'apprendimento automatico per addestrare in modo efficiente le reti neurali artificiali.
- Il suo obiettivo principale è **aggiornare i pesi e i bias della rete** in modo da **minimizzare la differenza tra l'output prodotto dalla rete e l'output desiderato**, attraverso lo **studio del gradiente dell'errore locale rispetto ai pesi**.
- Il processo prevede due fasi principali:
 - un passaggio in avanti (forward pass)
 - un passaggio all'indietro (backward pass).

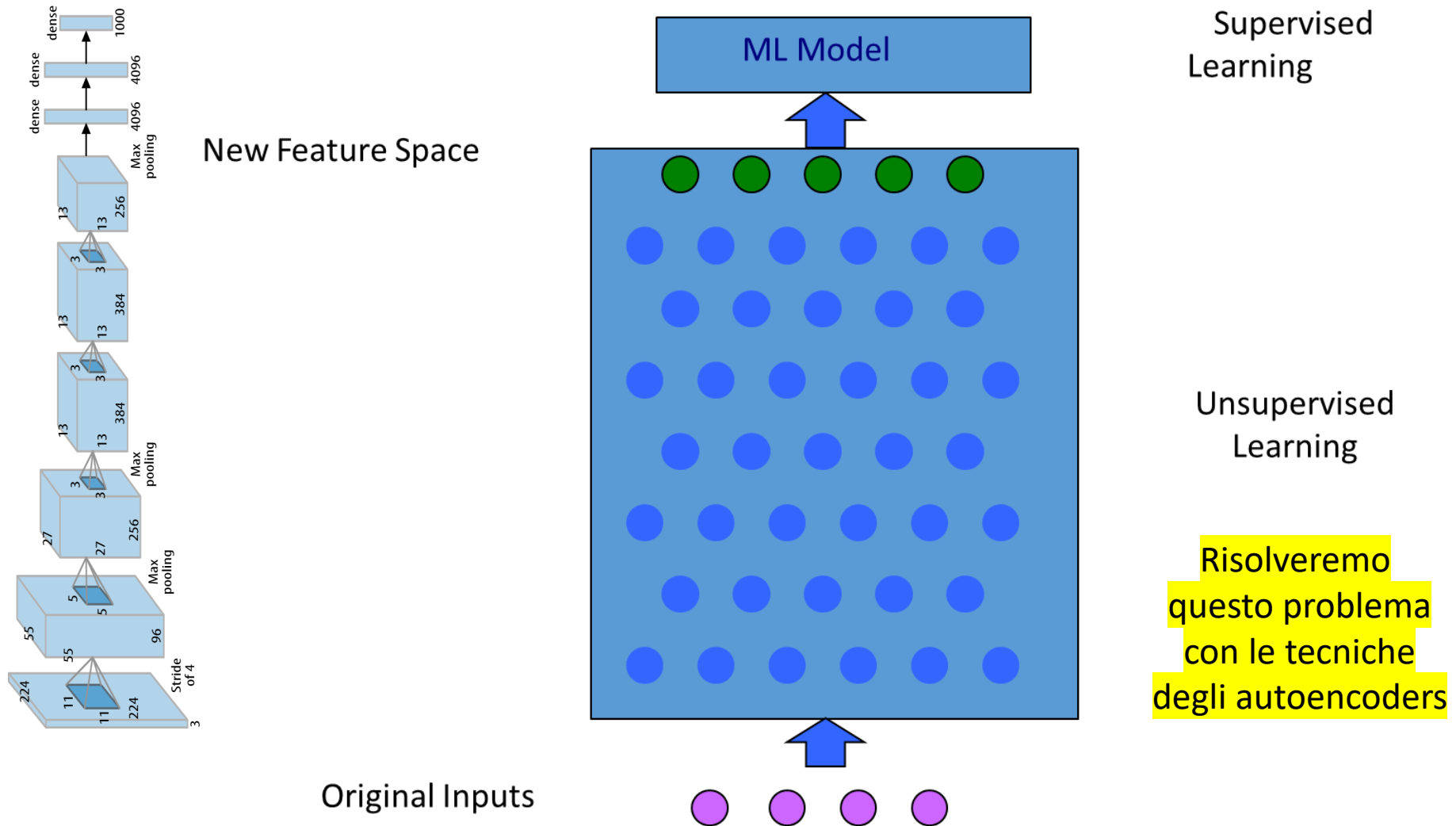


Limiti del gradiente nei primi strati delle reti deep

- Nei **primi strati della rete**, il gradiente può:
 - **annullarsi** → *vanishing gradient*
 - **crescere in modo incontrollato** → *exploding gradient*
- Questo rende l'aggiornamento dei pesi
 - molto lento o inefficace (vanishing)
 - numericamente instabile (exploding)
- Di conseguenza, i primi strati apprendono poco o nulla e faticano a catturare strutture significative dei dati. Per questo motivo sono state introdotte **tecniche alternative e correttive per i primi strati**

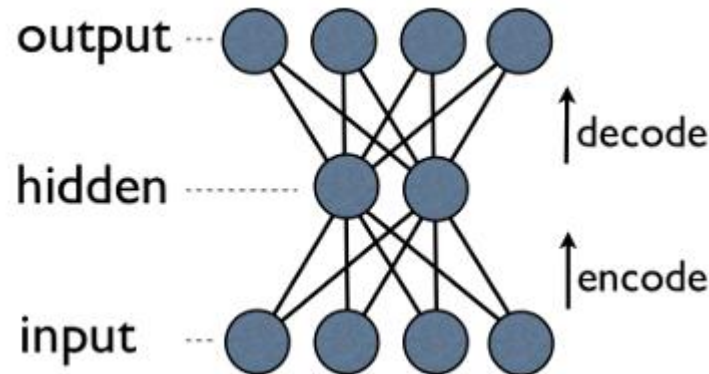


Deep Net with Greedy Layer Wise Training



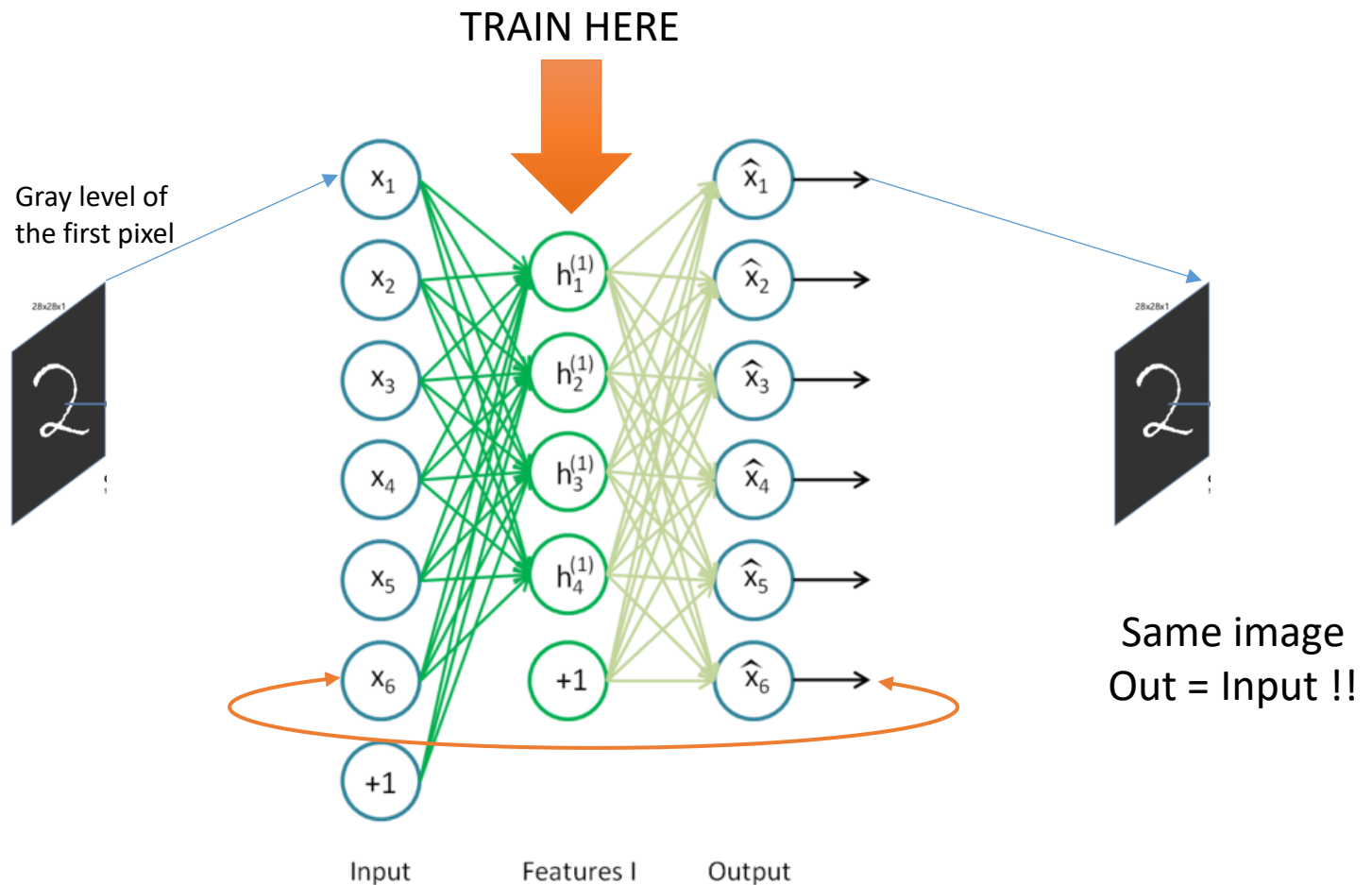
Autoencoders (encode->decode)

- Cercano di scoprire caratteristiche generiche dei dati.
- Imparano la **funzione identità** apprendendo sotto-caratteristiche importanti (non solo passando attraverso i dati).
- Compressione, ecc.
- Puoi usare solo nuove caratteristiche nel nuovo set di addestramento o concatenare entrambe.



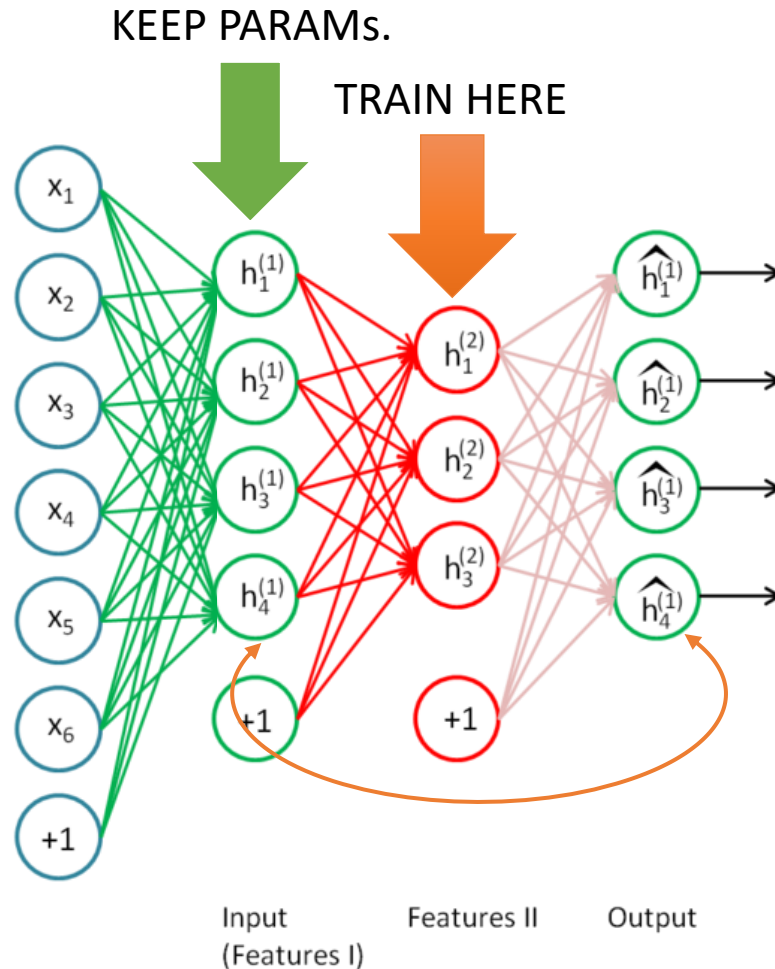
Stacked Auto-Encoders

- Stack many (sparse) auto-encoders in succession and train them using greedy layer-wise training
- Drop the decode output layer each time



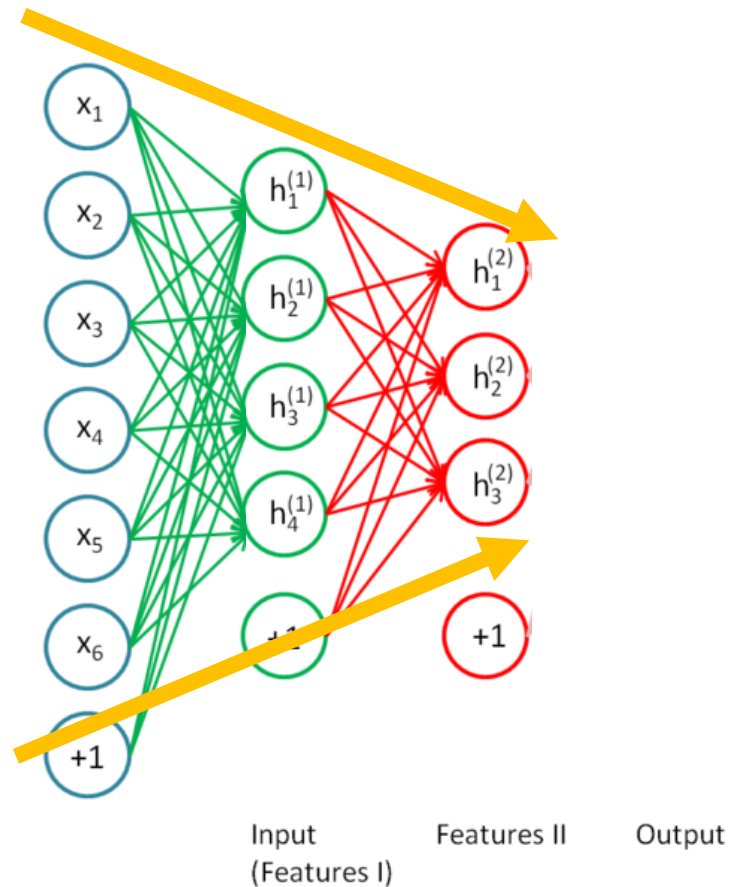
Stacked Auto-Encoders

- Stack many (sparse) auto-encoders in succession and train them using greedy layer-wise training
- Drop the decode output layer each time



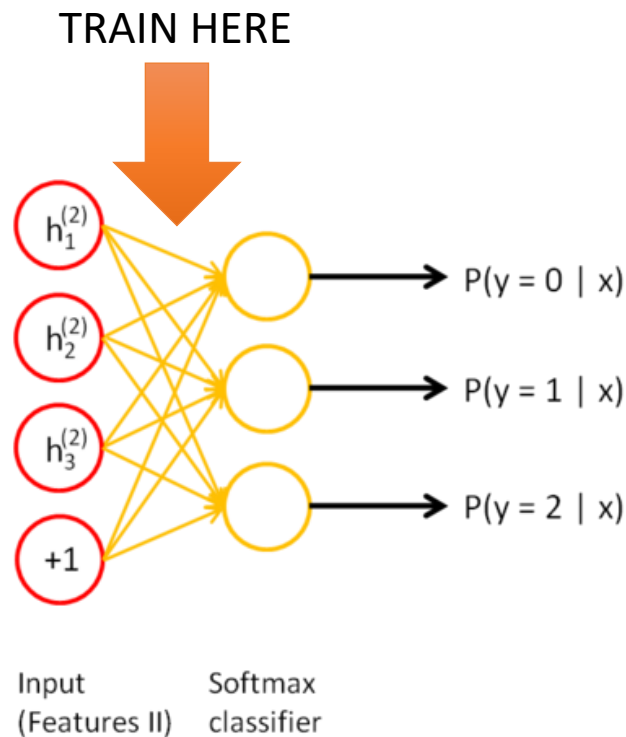
Stacked Auto-Encoders

- Tendere sempre a ridurre il numero di neuroni procedendo con gli strati



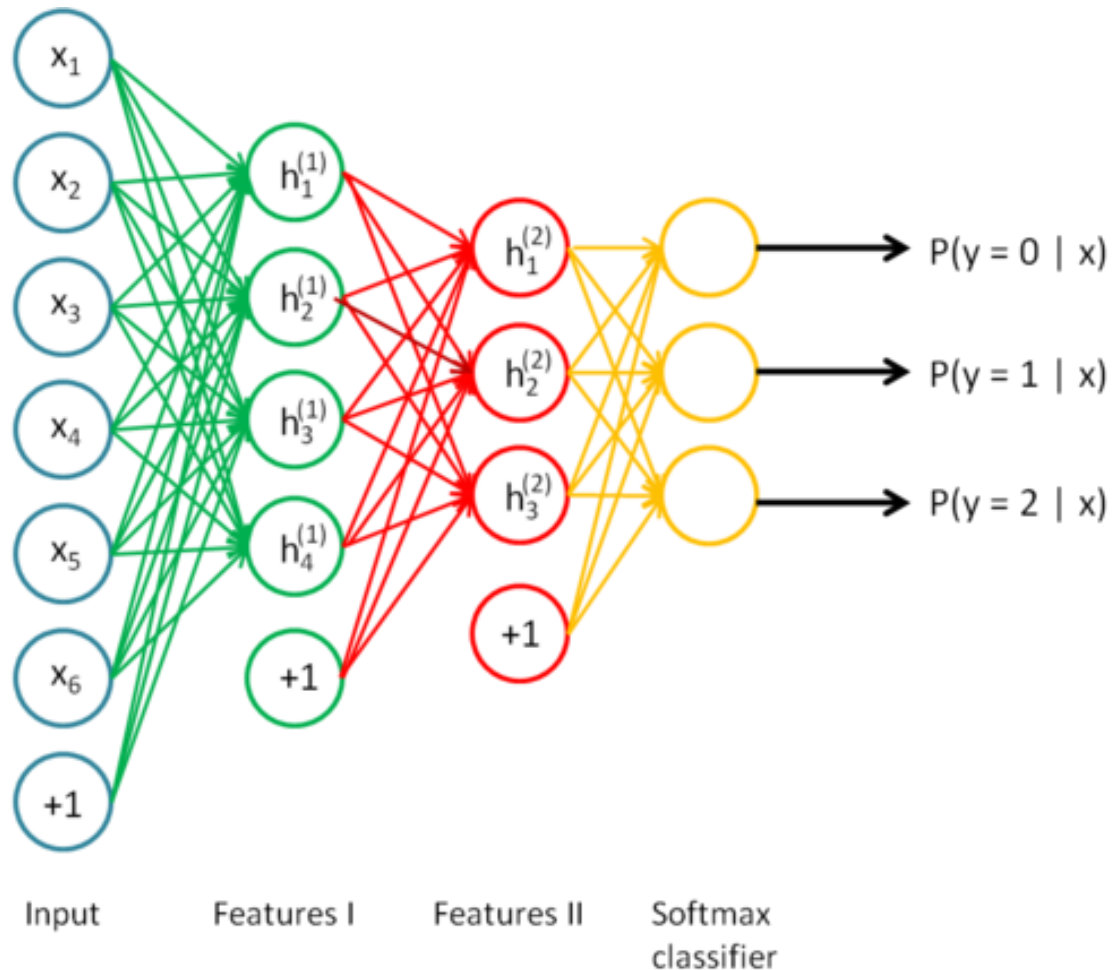
Stacked Auto-Encoders (2)

- Do supervised training on the last layer using final features

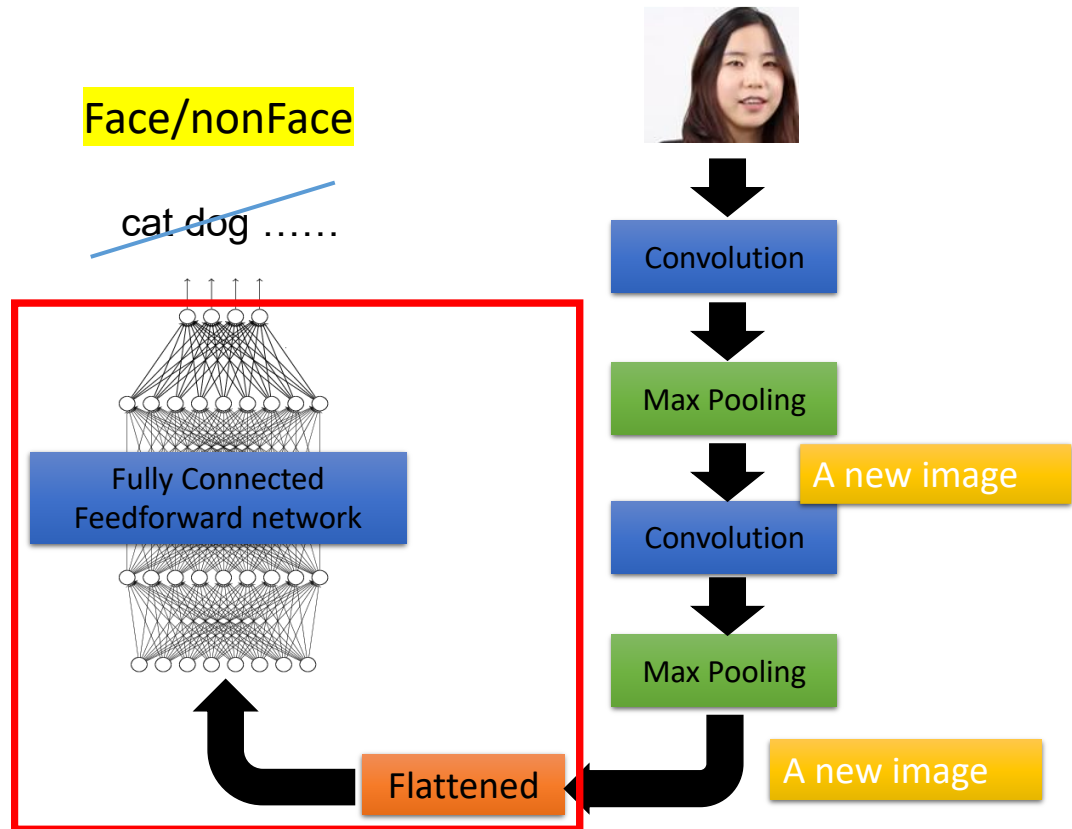


Stacked Auto-Encoders (3)

- Then do **supervised training on the entire network** to fine-tune all weights



Riallenare
con una back
propagation
sulla parte
fully
connected
per cambiare
output come
serve a noi



FINE TUNING

partendo da una CNN pubblica



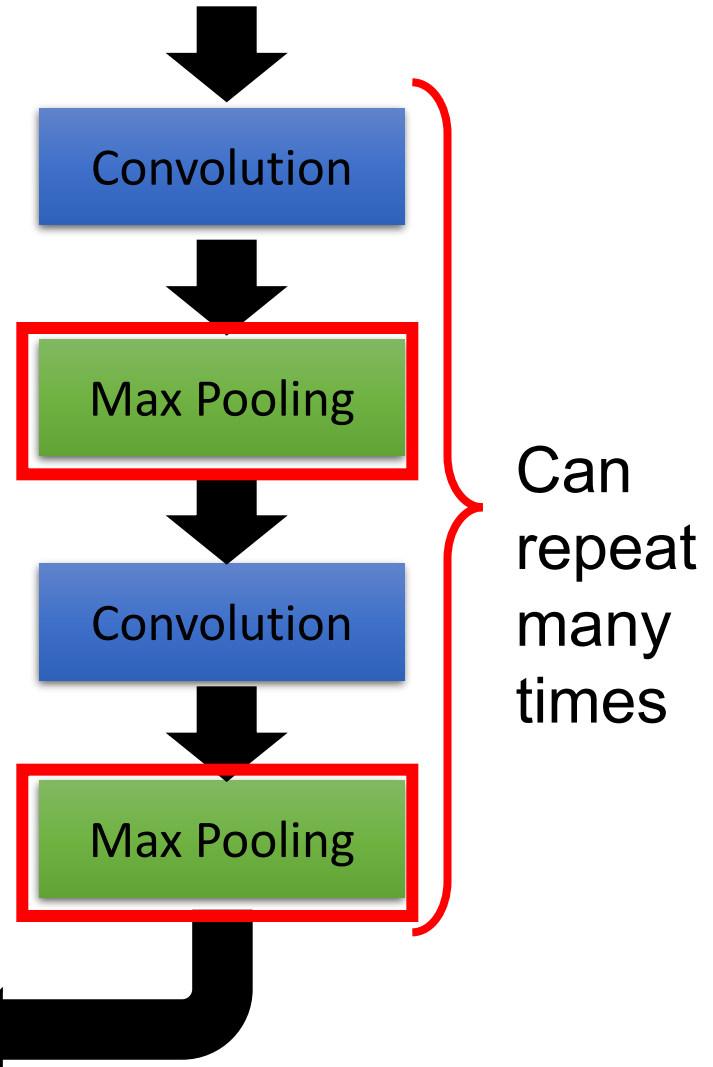
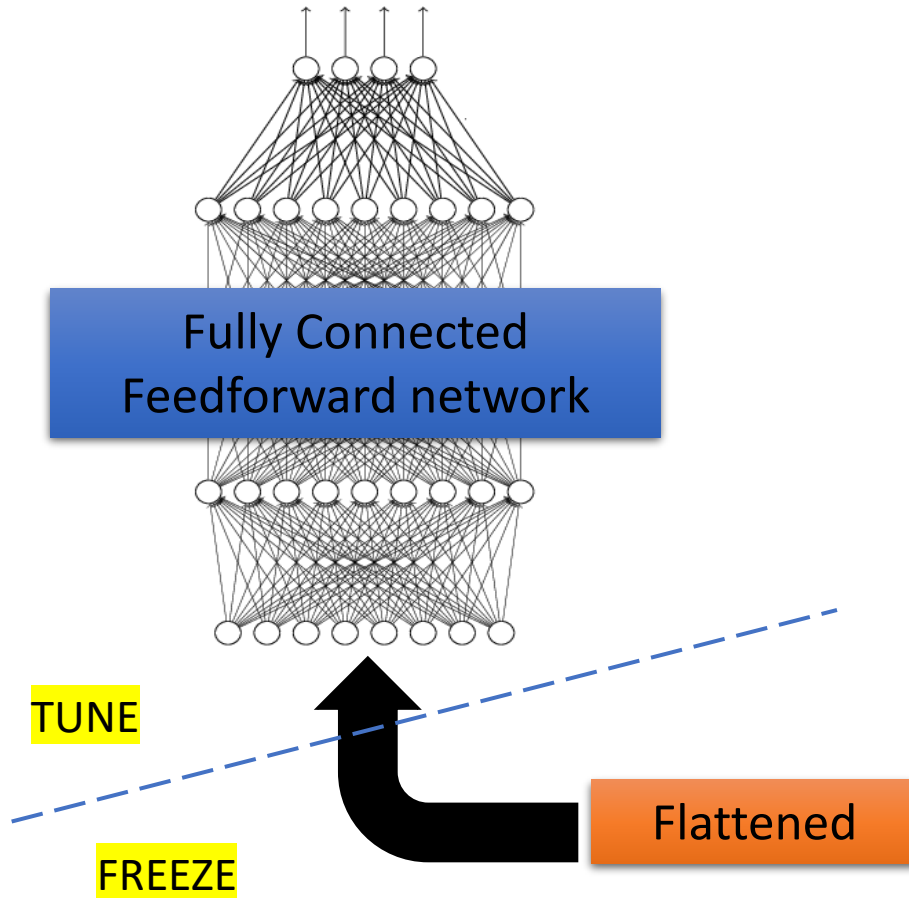
Classification PB1:

Face, body, dog,
horse, ...



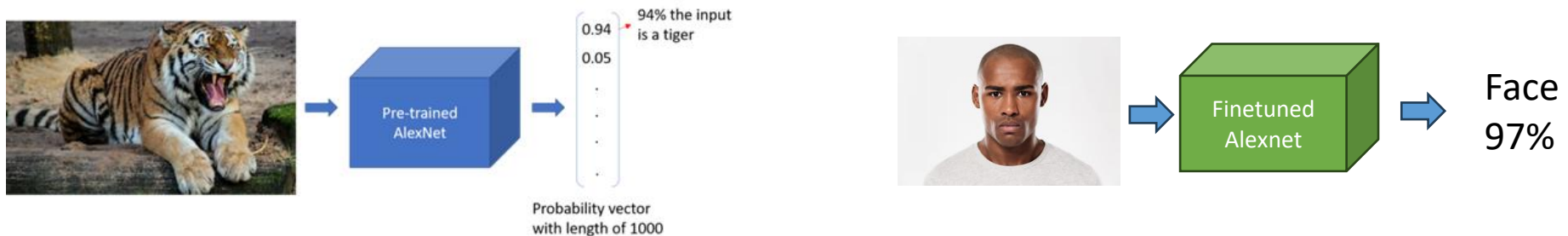
Classification PB2:

FACE YES/NO



Dal modello generico all'applicazione biometrica

- CNN sono nate per classificazione generica ma..
 - i dati biometrici hanno caratteristiche diverse
 - serve adattarne il funzionamento senza reinventare la loro struttura
- Il fine-tuning è la chiave pratica per uno sviluppatore (o medio-piccolo gruppo di ricerca)



Perché non (sempre) alleniamo tutto da zero?

- Dataset biometrici spesso limitati
 - Poche migliaia di esempi di immagini di persone non sono adeguati per allenare reti da zero di centinaia di milioni di parametri
- Addestramento completo è costoso
 - Possono servire settimane su architetture dedicate molto costose
- I primi strati apprendono pattern generici
 - Sono dei veri e propri estrattori generici, che non serve riallenare per un uso biometrico
- Riutilizzare è più robusto ed efficace
 - Abbiamo meno problemi di overfitting visto che abbiamo meno esempi di google, meta ecc. che le hanno prodotte

Ruolo delle CNN nel sistema biometrico

- La CNN non è (tutto) il sistema biometrico
- Produce rappresentazioni numeriche che usiamo per
 - prendere decisioni,
 - misurare,
 - confrontare
- Il matching avviene a valle di solito
- Le CNN sono diventate un componente molto importante
- Dalle loro prestazioni dipende fortemente il funzionamento di tutto il sistema

Ci sono rischi a usare modelli riallenati?

- **SI!**

Potrebbero esserci dei bias che non conosciamo ereditati dal dataset di allenamento iniziale!

- Per esempio potremmo essere meno bravi con alcune etnie, fasce d'età, etc..

- **SOLUZIONE:** tu designer dovrai con opportuni dataset di test

- Confrontare le performance per i diversi gruppi di utenti
- Verificare diverse condizioni applicative per controllare l'esistenza dei bias

Altri rischi a usare modelli riallenati?

SI!

- Scarsa separabilità inter-soggetto
- Sensibilità a fattori non biometrici
- Modelli non progettati per sicurezza
- Possibile leakage informativo
 - Es. estrae anche informazioni mediche non volute o sensibili
- Vulnerabilità a presentation attack e attacchi avversari

Adversarial mask designs evade facial recognition systems



Cosa significa *Leakage informativo*

- Se trovi in un file che dovrebbe essere usato per farti entrare in una area aziendale del tuo dipartimento di informatica trovi scritto che tu “*sei in forte sovrappeso e a rischio di infarto*”, sei contento?
- **I vettori biometrici posso nascondere informazione in modo meno esplicito!**
- **Leakage informativo nei vettori biometrici**
 - Il template contiene più informazioni del necessario
 - Informazioni non richieste né autorizzate
 - Difficili da controllare a posteriori
 - Rischio per privacy e sicurezza

Molto importante!

Privacy $\leftrightarrow$ Vettori estratti

- “Un buon template biometrico deve essere **utile per il matching o la misurazione richiesta**, ma **inutile per qualsiasi altra cosa.**”
- Le reti ci ritornano dei vettori di caratteristiche che gestiamo nel sistema, ma non dovremmo essere in grado di estrarre informazioni di tipo non autorizzato dagli utenti
 - Gender, orientamenti, predisposizioni a malattie, profilazione non autorizzata, ecc.
- Vero in teoria, **difficile da evitare nella pratica**
 - Vedremo esempi e soluzioni nelle lezioni, ma occorre esserne consapevoli nella gestione di queste informazioni.
 - Sembrano vettori casuali e anonimi.... Non lo sono!)

Dispensa introduttiva

Per approfondire i temi sul Deep Learning presentati, si consiglia la consultazione della breve dispensa focalizzata sull'ambiente Matlab (completo e semplice da utilizzare)

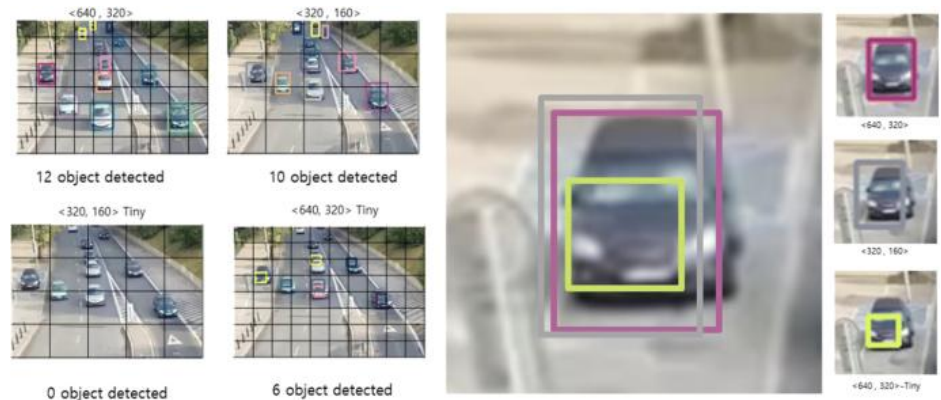
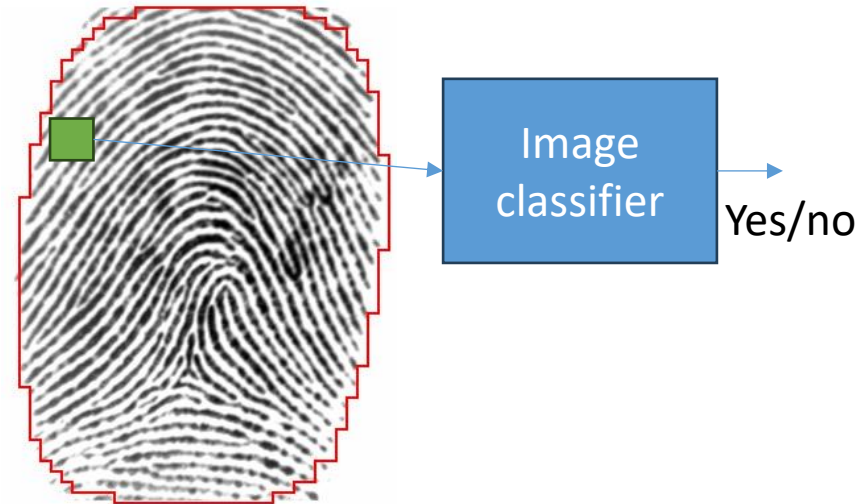


Direct image classification

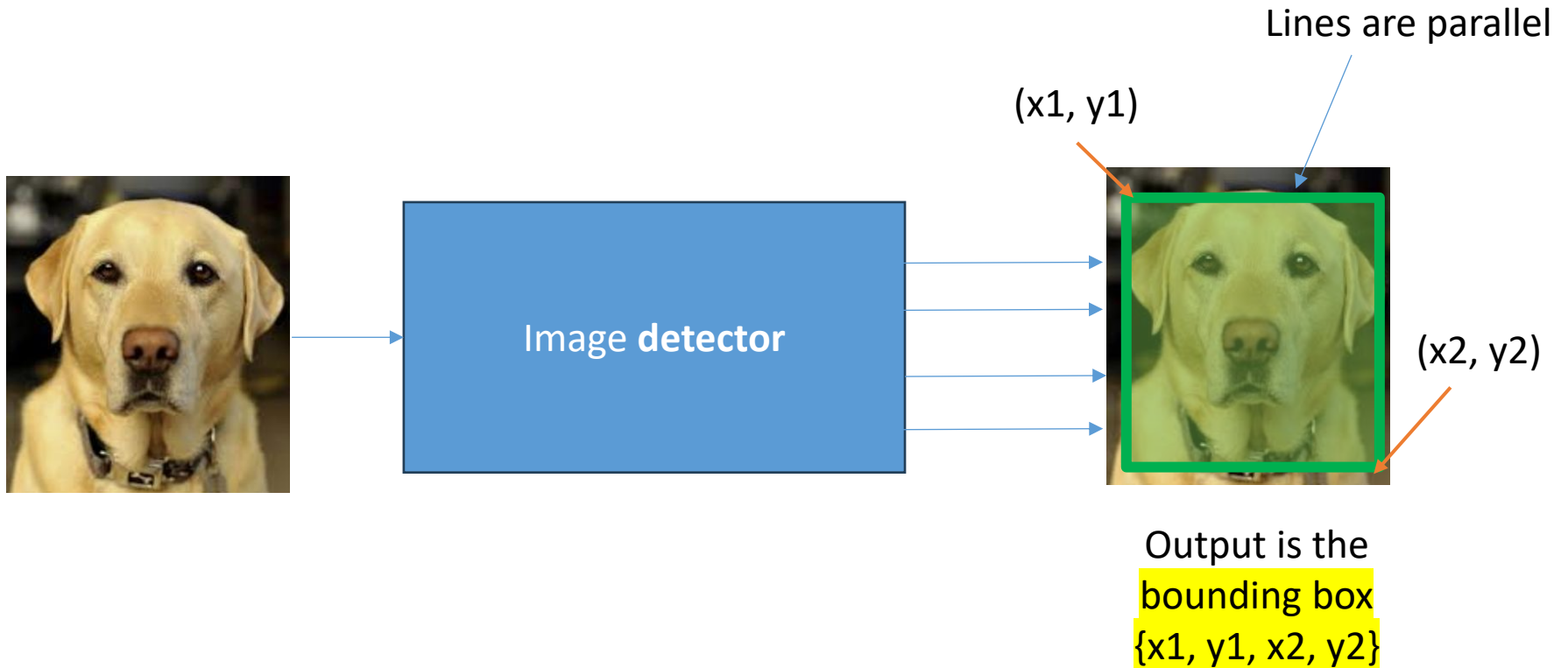


Direct image **classification** uso classico in biometria

- Molto veloce,
per sistemi embedded
- Segmentazione
block processing
Volto, Impronte
- Oggetti in scene
di sorveglianza
via block processing
o blob detector



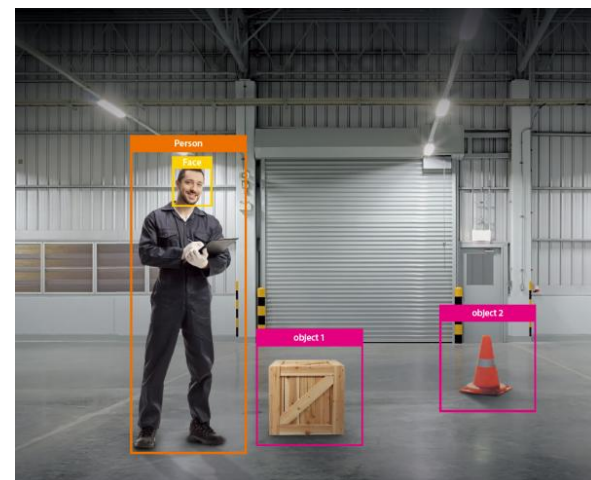
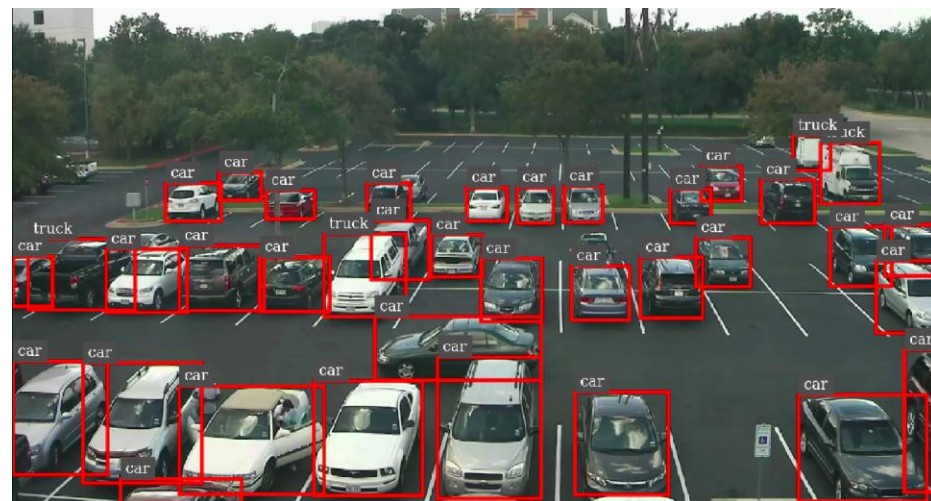
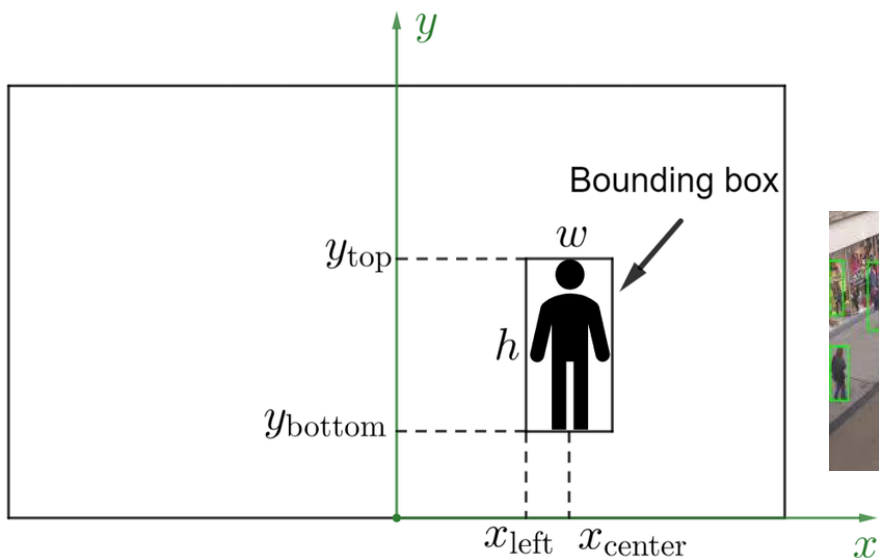
Object detection bounding box



It's almost impossible to get measurements

Object detection: bounding box

- Realizzato con reti piu' grandi e quindi richiede maggiore potenza computazionale
- In ingresso alla rete si ha una immagine intera o porzioni maggiori



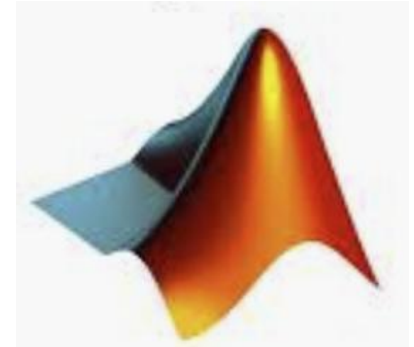
Laboratorio

Gli strumenti da usare



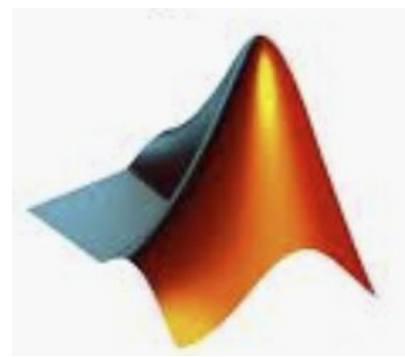
TOOLS

- Alcune note solo per PREPARARE i toolbox per le prossime lezioni. Rivedremo in dettaglio gli strumenti nelle prossime lezioni.
- Strumenti principali utilizzati nel corso:
 - Matlab (moltissimi strumenti e funzioni)
 - Colab (server online, TPU, ...)
 - Keras (la libreria Python per l'apprendimento profondo)
- Offrono il miglior compromesso tra usabilità e complessità, e una buona curva di apprendimento. Supportano Keras e molti modelli di reti neurali pre-addestrati.



colab



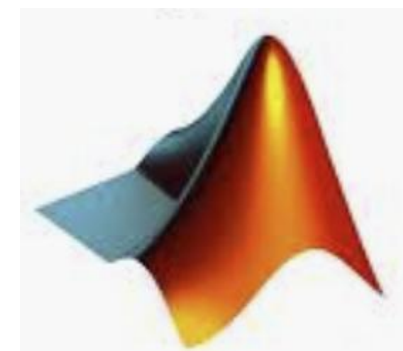


TOOL#1: Matlab

- Matlab è il primo strumento nel nostro corso. Non solo apprendimento automatico... ma molto, molto di più. Migliaia di funzioni ben documentate e potenti.
- Leggi le istruzioni e scarica la tua licenza gratuita di Matlab come studente UNIMI.
- Scarica l'ultima versione! Le toolbox stanno migliorando rapidamente!
https://work.unimi.it/servizi/servizi_tec/79539.htm
(o semplicemente cerca su Google >> UNIMI MATLAB)
- Vedi le istruzioni per installare MATLAB su una **macchina non connessa** a Internet.

TOOL#1:

Matlab Tooboxes



Installa tutte i toolbox, ma se hai poco spazio sul disco rigido del tuo PC / banda larga ridotta, installa solo le principali utili per il corso come:

- **Apprendimento profondo (Deep Learning)**
- Logica fuzzy (Fuzzy Logic)
- Statistica e apprendimento automatico (Statistics and Machine Learning)
- Elaborazione delle immagini (Image Processing)
- Elaborazione dei segnali (Signal Processing)
- **Visione artificiale (Computer Vision)**

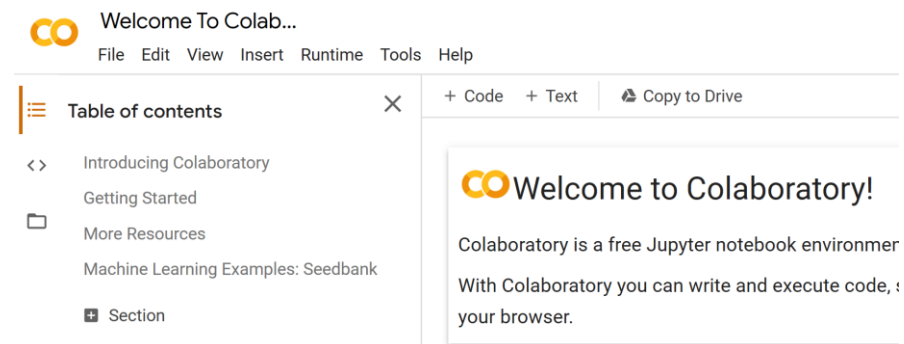
Oppure scegline alcuni da utilizzare in futuro per i tuoi progetti, è gratuito! ...

TOOL#2:

Google Colab

COLAB = Colaboratory è un ambiente gratuito per notebook Jupyter che non richiede alcuna configurazione e funziona interamente nel cloud. Puoi:

- scrivere ed eseguire codice,
- salvare e condividere le tue analisi,
- e accedere a potenti risorse di calcolo, tutto gratuitamente dal tuo browser.



TOOL#2:

Google Colab



Google

Sign in

Use your Google Account

Email or phone

COLAB È molto, molto utile!

Ci aiuta ad ottimizzare il tempo, raggiungendo i nostri obiettivi senza perdere tempo in tecnicismi.

Ma poiché richiede un account Google per salvare dati e codice, non è strettamente obbligatorio per il corso.

- Hai altre opzioni (meno facili e pronte all'uso):
 - utilizza Matlab per tutto
 - scarica altri strumenti locali (probabilmente ti serviranno GPU) come Jupyter Notebook, Anaconda, Python, ecc.
 - puoi creare/usare un specifico account Google senza alcuna informazione personale memorizzata al suo interno.
 - Non chiedere però assistenza...

TOOL#3 (forse Colab è preferibile per noi)



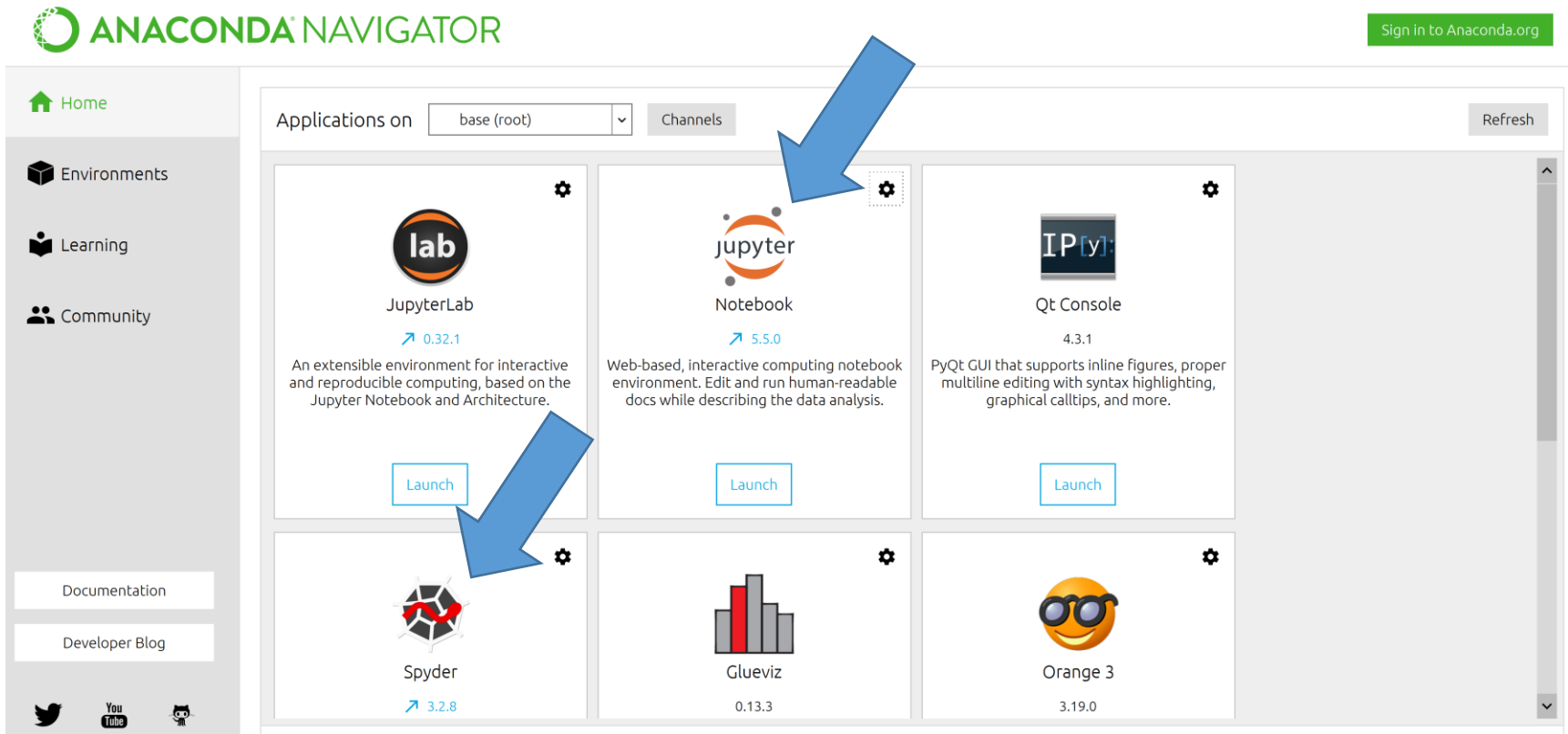
Uno strumento potente per il calcolo scientifico e l'analisi dei dati che semplifica la gestione dei pacchetti e il loro dispiegamento, rendendolo una scelta popolare tra scienziati dei dati, ricercatori e sviluppatori.

- Distribuzione di Python e R.
- Molte librerie e strumenti preinstallati comunemente utilizzati in data science, apprendimento automatico e altre applicazioni scientifiche.
- Ottimo sistema di gestione dei pacchetti (facilità di installazione, aggiornamento e gestione di diverse versioni di pacchetti e dipendenze).
 - Per installare modelli/reti complessi e relative funzioni avere pacchetti si auto-installano è molto molto utile

Anaconda Navigator

L'applicazione desktop ti permette di gestire facilmente applicazioni integrate, pacchetti e ambienti senza utilizzare la riga di comando e in locale. Per esempio

- Un notebook simile a Colab
- Un IDE per programmare meglio



Jupyter Notebook



localhost:8888/notebooks/Untitled.ipynb



jupyter Untitled Last Checkpoint: alcuni secondi fa (autosaved)

File Edit View Insert Cell Kernel Widgets Help

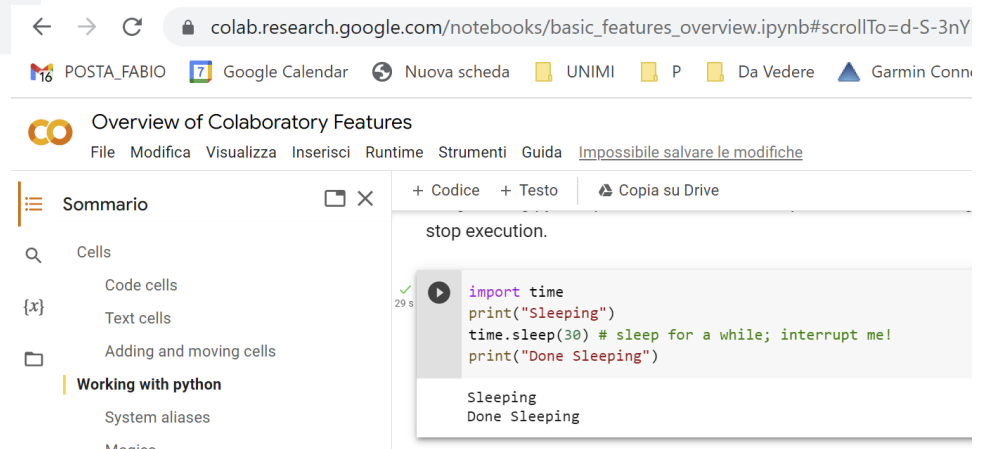
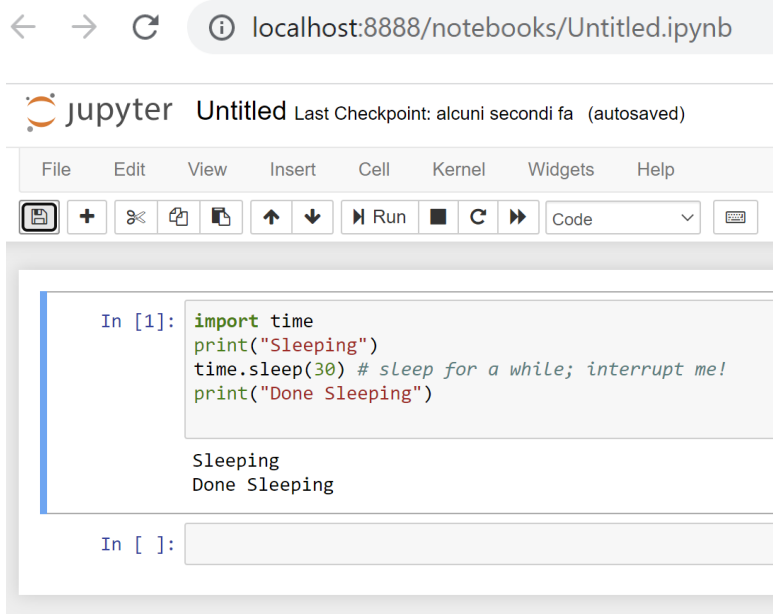


```
In [1]: import time
print("Sleeping")
time.sleep(30) # sleep for a while; interrupt me!
print("Done Sleeping")
```

Sleeping
Done Sleeping

In []:

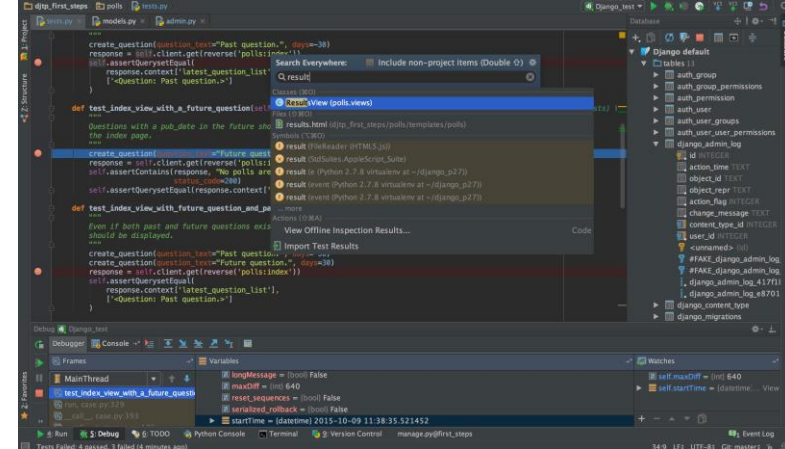
Jupyter Notebook and Colab



Google Colab is ready and more hardware-independent (e.g., test your code on TPUs etc..) with some limitations which are not so relevant in this course

Or use an IDE

(Integrated Development Environment)



PyCharm

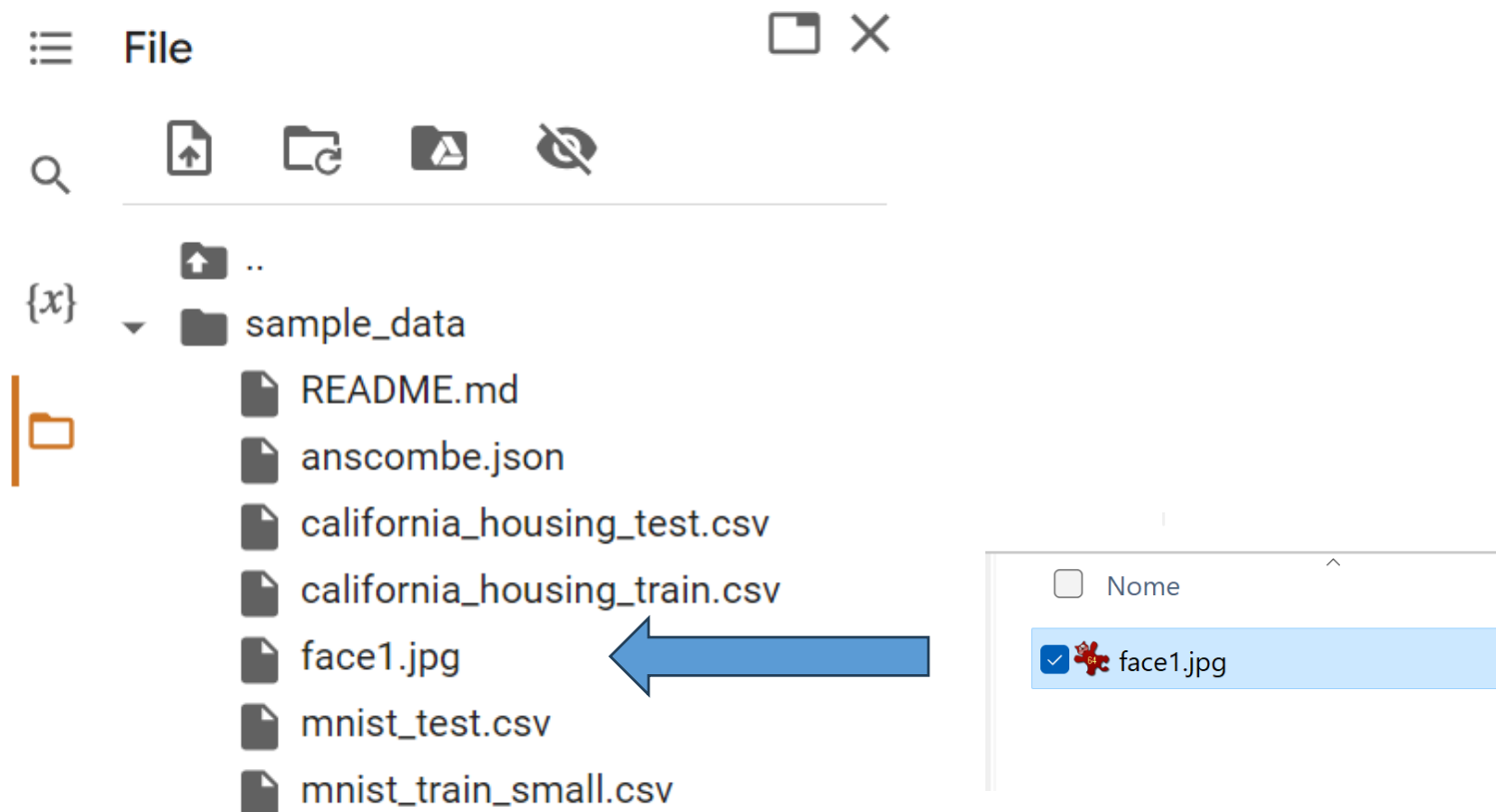
The Python IDE
for Professional Developers

JET
BRAINS

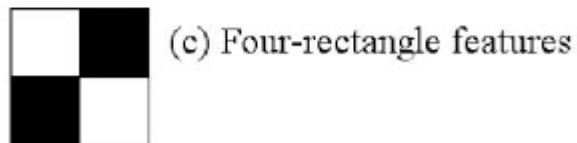
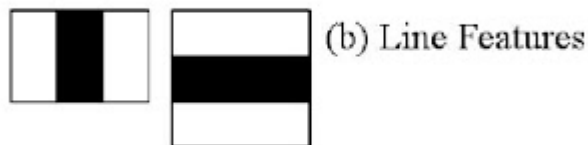
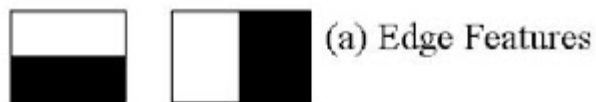


spyder

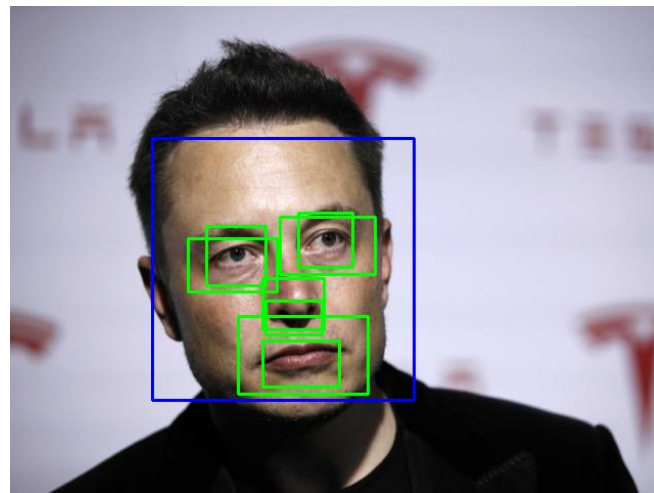
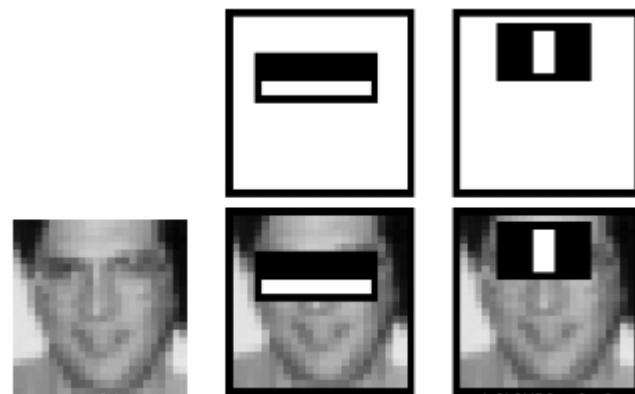
Caricare il file da elaborare



Modello di ML allenato (non deep): Haar cascade



Il modello Haar Cascade di OpenCV è un algoritmo di machine learning utilizzato per il rilevamento di oggetti in immagini e video. Nel contesto della rilevazione del viso, l'algoritmo individua le regioni dell'immagine che più probabilmente contengono un volto e le segnala come coordinate di una bounding box (rettangolo che delimita a queste regioni)



Aprire un nuovo blocco e scrivere/copiare il codice

Step 1: Install OpenCV (In Google Colab, run it as a shell command with !)

```
!pip install opencv-python
```

OpenCV è una libreria Intel che è utilissima ed efficiente

Step 2: Import Libraries

```
import cv2
```

```
from google.colab.patches import cv2_imshow
```

Aprire un nuovo blocco e scrivere/copiare il codice

```
# Step 3: Load Haar Cascade Classifier
```

```
face_cascade =  
cv2.CascadeClassifier(cv2.data.harcascades +  
'haarcascade_frontalface_default.xml')
```

Il modello prealllenato viene caricato
nella versione per i volti frontali

```
# Step 4: Read Image
```

```
# Upload an image to Google Colab and read it
```

```
img = cv2.imread('./sample_data/face3.jpg')
```

Aprire un nuovo blocco e scrivere/copiare il codice

Step 5: Perform Face Detection

Funziona con il canale grigio

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.1, 4)
```

Step 6: Draw Rectangles

Produce più possibili bounding box se vi sono più immagini

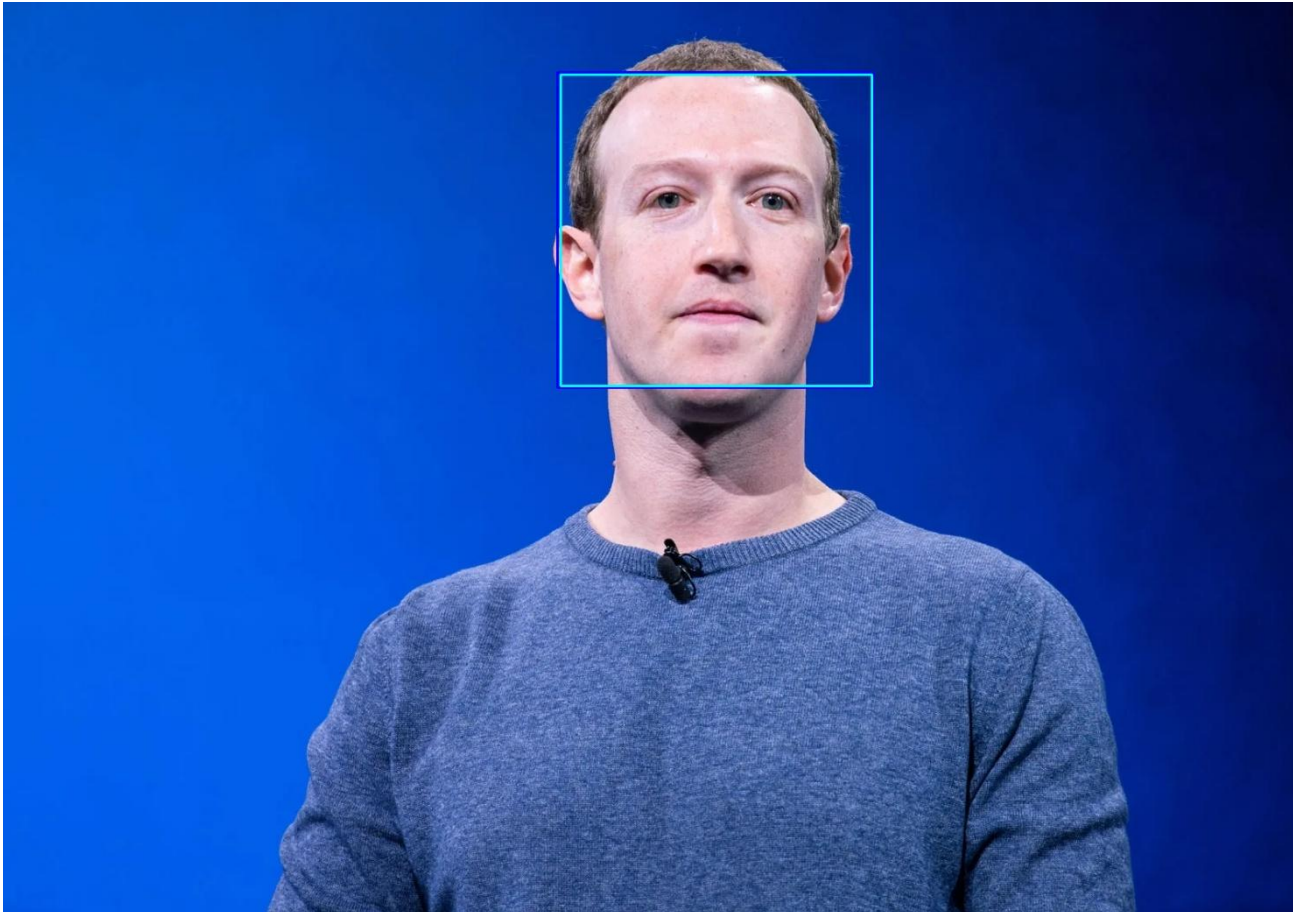
```
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (255, 255, 0), 2)
```

Step 7: Display the Image

```
cv2_imshow(img)
```

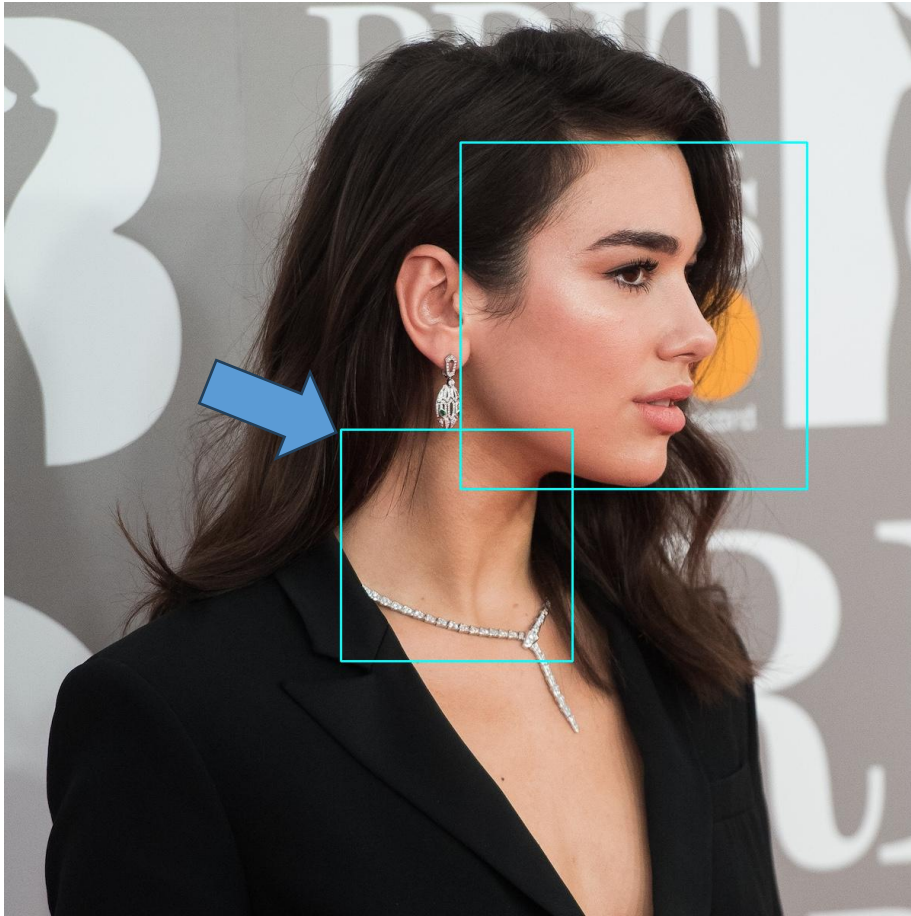
Output

face1.jpg



Output

face2.jpg

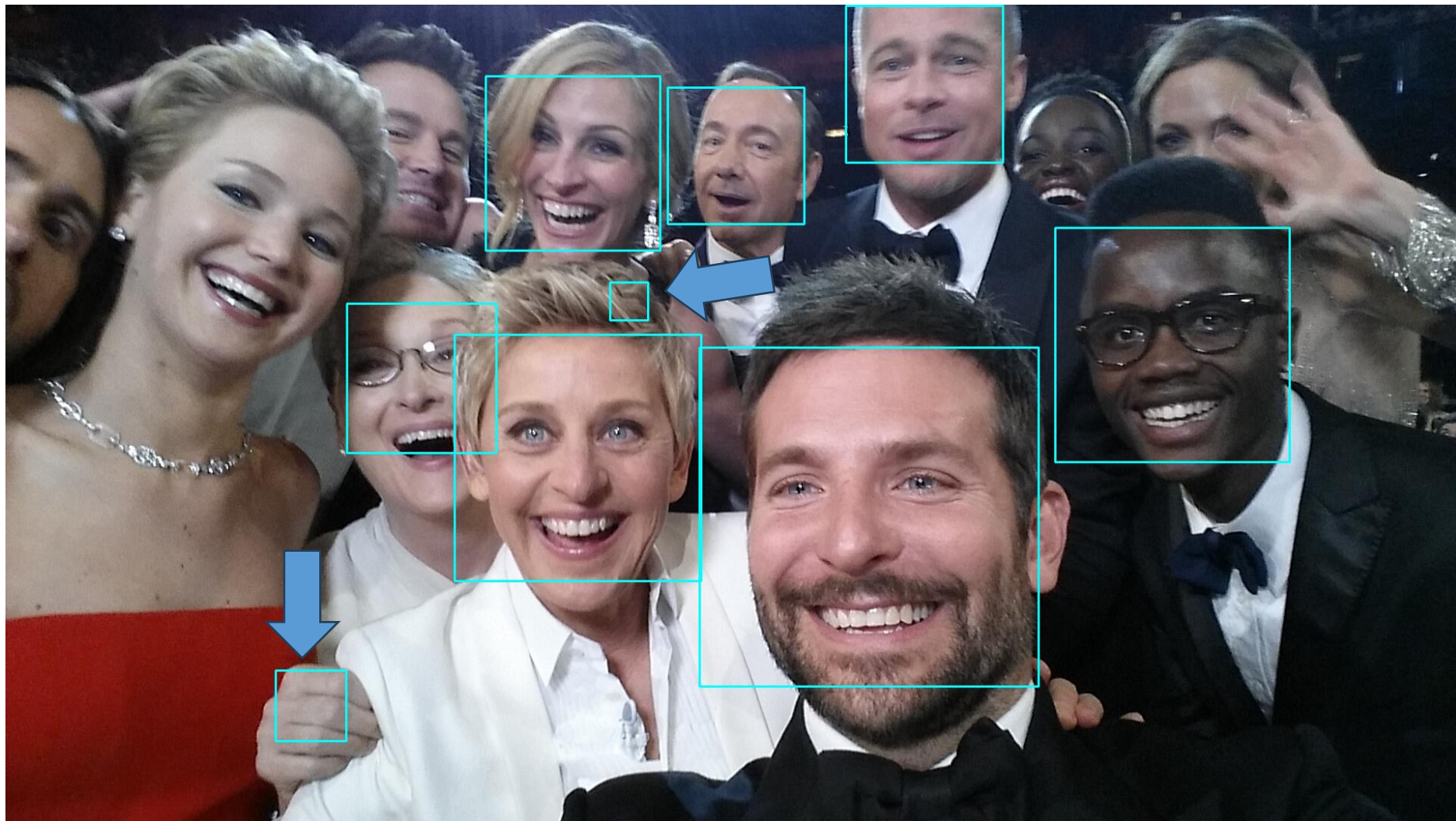


Nella semplicità del modello, pur non essendo stato allenato con volti laterali, abbiamo una buona detection

Provoca il primo falso positivo ma potrebbe essere filtrato successivamente con altre tecniche che vanno a misurare la qualità del voto

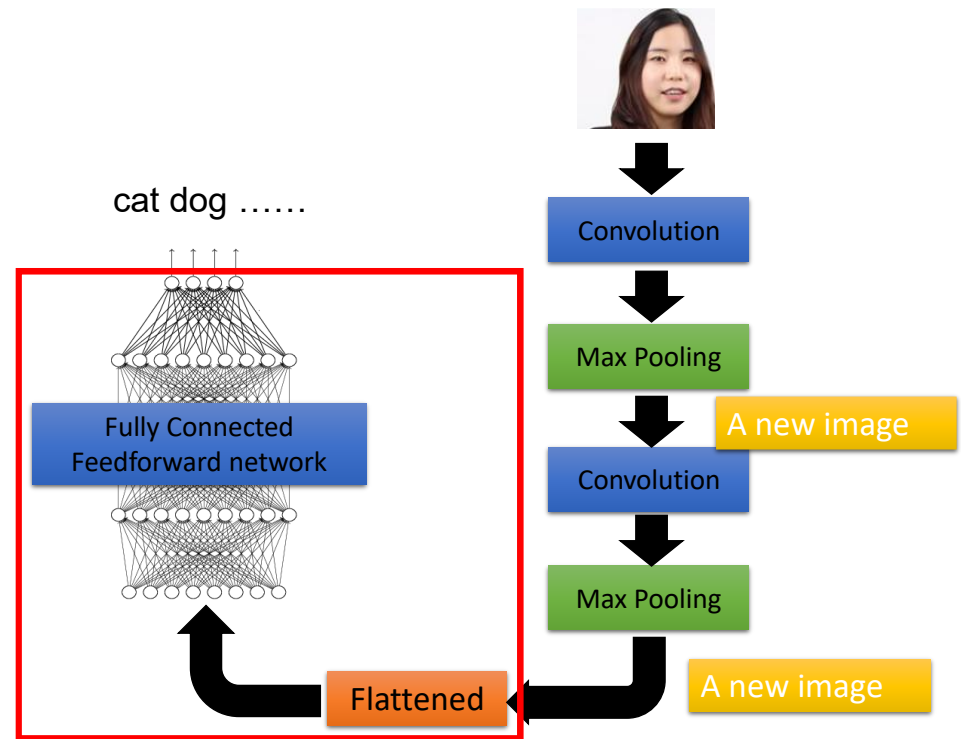
Output

face3.jpg



Abbiamo visto

- Struttura
- Funzionamento degli strati convoluzionali
- Apprendimento
 - Non supervisionato
 - Supervisionato
- Fine tuning
- Rischi di privacy associato ai vettori biometrici



Appendici con il codice

Non richiesto all'esame

```
# Step 1: Install OpenCV (In Google Colab, run it as a shell command with !)
!pip install opencv-python

# Step 2: Import Libraries
import cv2
from google.colab.patches import cv2_imshow

# Step 3: Load Haar Cascade Classifier
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade_frontalface_default.xml')

# Step 4: Read Image
# Upload an image to Google Colab and read it
img = cv2.imread('./sample_data/face3.jpg')

# Step 5: Perform Face Detection
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.1, 4)

# Step 6: Draw Rectangles
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (255, 255, 0), 2)

# Step 7: Display the Image
cv2_imshow(img)
```